



Università degli Studi di Ferrara

UNIVERSITÀ DEGLI STUDI DI FERRARA
CORSO DI LAUREA IN INFORMATICA

*Envass 2.0: un sistema gestionale per
l'analisi microbiologica*

Relatore:

Prof. Guido SCIAVICCO

Laureando:

Christian MICK

ANNO ACCADEMICO 2025 – 2026

Indice

	Page
1 Introduzione	5
2 Background	7
2.1 Evoluzione del Web e Tecnologie Client-Side	8
2.1.1 Il linguaggio di marcatura HTML	8
2.1.2 La formattazione tramite CSS	9
2.1.3 Framework per il Design Responsivo: Bootstrap	10
2.1.4 Interattività tramite JavaScript	11
2.2 Sviluppo Lato Server e Linguaggio PHP	13
2.2.1 Il Modello Architettuale Client-Server	13
2.2.2 Il Protocollo di Comunicazione HTTP	14
2.2.3 Evoluzione e Paradigmi del Linguaggio PHP	16
2.2.4 Gestione delle Dipendenze tramite Composer	18
2.3 Architettura del Framework Laravel	19
2.3.1 Il Pattern Architettuale MVC	20
2.3.2 Il Ciclo di Vita della Richiesta	22
2.3.3 Service Container	24
2.4 Persistenza dei Dati e Database Relazionali	25
2.4.1 Sistemi di Gestione di Database	25
2.4.2 Eloquent ORM	26

2.4.3	Versionamento del Database: Migrations	27
3	Progetto ENVASS	29
3.1	Ente di riferimento: CIAS	29
3.2	Descrizione del progetto	30
3.3	Anagrafica	31
3.4	Liste	32
3.5	Verbali	33
3.6	Analisi	34
3.7	Rapporti di prova	34
3.8	Sistema di logging	35
4	Implementazione del progetto	37
4.1	Tecnologie utilizzate	38
4.2	Routing	39
4.2.1	Utenti, ruoli e permessi	39
4.2.2	Middleware del login	41
4.2.3	Rotte	42
4.3	MVC	42
4.4	Automatismo	48
4.4.1	Campioni	50
4.4.2	PDF	50
4.4.3	Blocco del rapporto di prova	53
4.5	Migrazione dati dalla versione 1 alla versione 2	53
4.6	Logging	53
5	Conclusione	63

Capitolo 1

Introduzione

Scrivi qua l' introduzione

Background

Questo capitolo presenta i concetti teorici e le tecnologie che costituiscono la base dello sviluppo del sistema realizzato. L'obiettivo è fornire una panoramica degli strumenti, dei paradigmi e delle metodologie utilizzate durante il progetto, introducendo le principali caratteristiche e il ruolo che ciascuna tecnologia ricopre nell'architettura dell'applicazione.

Nella prima sezione si illustrano i concetti fondamentali relativi al frontend, con particolare attenzione agli aspetti legati all'interfaccia utente, alla presentazione visiva e all'interazione dinamica tra utente e sistema. Successivamente vengono presentati i principi dell'architettura lato server e il linguaggio PHP, descrivendone il modello di comunicazione e gli strumenti utilizzati per lo sviluppo e la gestione delle dipendenze software.

In seguito si analizza Laravel, framework basato su PHP di cui vengono approfonditi il paradigma *Model-View-Controller* (MVC) e i meccanismi di gestione del ciclo di vita delle richieste e dei componenti del sistema. Infine, si introducono i sistemi database, esaminando i principi di organizzazione dei dati e gli strumenti per la gestione e l'evoluzione dello schema della base di dati.

La comprensione degli argomenti trattati in questo capitolo risulta fondamentale per contestualizzare le scelte progettuali e implementative descritte nei capitoli

successivi.

2.1 Evoluzione del Web e Tecnologie Client-Side

L'evoluzione del Web client-side si fonda sulla sinergia di tre tecnologie principali: HTML per la struttura, CSS per la presentazione e JavaScript per il comportamento.

2.1.1 Il linguaggio di marcatura HTML

Il *Hypertext Markup Language* (HTML) è il componente fondamentale del Web, che definisce il linguaggio e la struttura dei contenuti disponibili in rete. Lo scopo del linguaggio HTML è quello di strutturare e definire semanticamente le informazioni. La sua evoluzione è contraddistinta da differenti fasi che corrispondono a rilasci statici e definiti (come l'HTML4 o l'XHTML), ma ad oggi le specifiche HTML sono mantenute come uno "standard vivente" (Living Standard) mantenuto dal gruppo *Web Hypertext Application Technology Working Group* (WHATWG). La particolarità del modello prevede un aggiornamento continuo delle specifiche senza l'utilizzo di rilasci numerati. [1] Alla base c'è il concetto di *Hypertext* (ipertesto) che definisce la struttura di link (collegamenti) permettendo la navigazione tra diverse pagine web o all'interno dello stesso documento. I link rappresentano una componente essenziale della rete. L'architettura del *World Wide Web* (WWW) si fonda sulla pubblicazione di contenuti e sulla loro interconnessione tramite link.

L'HTML non è un linguaggio di programmazione, bensì un linguaggio di marcatura, in quanto utilizza una sintassi basata su *tag* (etichette racchiuse tra parentesi angolari < >) per istruire il browser su come interpretare e visualizzare i dati. Ad esempio, il *tag* <h1> definisce un'intestazione principale, il *tag* <p> delimita un paragrafo, e il *tag* <a> (denominato *Anchor*) abilita la funzione ipertestuale, permettendo di collegare risorse informative differenti all'interno del WWW.

Quando un browser riceve un documento HTML, ne genera una rappresentazione interna denominata *Document Object Model* (DOM). Il DOM è una struttura gerarchica ad albero che mappa tutti gli elementi della pagina, rendendoli accessibili e manipolabili tramite linguaggi di scripting. Ogni documento segue una struttura ben definita in cui gli elementi possono essere annidati. I contenuti visi-


```
/* 1. Anatomia di una regola CSS singola */
h1 {
    color: #2c3e50;
    font-size: 24px;
}

/* 2. Raggruppamento di piu selettori per
ottimizzazione del codice */
h1, h2, p {
    font-family: 'Helvetica', sans-serif;
    margin-bottom: 15px;
}
```

Figura 2.1: Esempio di regole CSS: anatomia di una regola e raggruppamento dei selettori.

bili all'utente sono racchiusi all'interno del *tag* `<body>`, mentre le informazioni di servizio, i metadati o i riferimenti a file esterni risiedono nel *tag* `<head>`. Infine, per differenziare e raggruppare elementi specifici, l'HTML fornisce attributi globali come `id` (per identificare in modo univoco un singolo elemento) e `class` (per classificare gruppi concettuali di elementi), che fungono da fondamentali punti di aggancio per stili e script. Gli elementi appena descritti permettono all'HTML di definire lo scheletro informativo della pagina, delegando però gli aspetti visivi e comportamentali ad altri linguaggi specializzati [2, cap. 5].

2.1.2 La formattazione tramite CSS

I *Cascading Style Sheets* (CSS) rappresentano il linguaggio dedicato esclusivamente alla formattazione ed all'aspetto visivo dei documenti *web*. Se il HTML fornisce la semantica e i contenuti, il CSS interviene per garantire una rigorosa “separazione” tra i contenuti stessi e la loro presentazione esteriore. Questo paradigma consente il controllo centralizzato dell'aspetto visivo, permettendo di definire in modo totalmente indipendente e granulare dettagli cruciali come i colori, la tipografia, gli spazi (margini e *padding*) e la disposizione degli elementi sullo schermo.

Il linguaggio CSS si organizza attraverso un sistema di regole che collegano gli elementi di una pagina *web* al loro aspetto estetico. Ogni regola inizia con un selettore, ovvero il termine che indica al *browser* quali parti del testo o della pagina devono essere modificate. Dopo il selettore si trova la dichiarazione, scritta tra parentesi graffe, che contiene i dettagli pratici dello stile. Questa si divide a sua volta in una proprietà, che sceglie la caratteristica da cambiare come il colore o la grandezza, e in un valore, che stabilisce la modifica effettiva. Per rendere il lavoro più veloce e il codice più pulito, è possibile unire più selettori insieme in modo da assegnare lo stesso stile a diversi elementi contemporaneamente con un'unica istruzione [3] .

Un vantaggio architetturale fondamentale del CSS è la possibilità di raccogliere tutte le regole all'interno di file esterni dotati di estensione `.css`. Un documento HTML può collegarsi a questi file esterni utilizzando il *tag* `<link>` inserito nel suo `<head>`. Questa tecnica consente a decine o centinaia di pagine *web* di condividere e puntare al medesimo foglio di stile; di conseguenza, una singola modifica apportata al file CSS si ripercuote istantaneamente sull'intero sito *web*, facilitando scalabilità, coerenza visiva ed operazioni di manutenzione [2, ch. 11].

2.1.3 Framework per il Design Responsivo: Bootstrap

L'ambiente in cui il *Web* viene fruito oggi è estremamente frammentato: i contenuti devono essere visualizzati su schermi di dimensioni diverse, dai grandi monitor *desktop* ai *tablet*, fino ai piccoli *display* degli *smartphone*, nonché su dispositivi assistivi o degli schermi televisivi. La necessità di risolvere questo problema ha imposto l'adozione del “*Design Responsivo*” e di filosofie progettuali come il “*Mobile First*”, introdotto da specialisti del settore come Luke Wroblewski, che spingono per ottimizzare i siti *web* primariamente per i dispositivi mobili prima di adattarli agli schermi più ampi [2, ch. 3]. L'implementazione di questi paradigmi invece è molto complessa. Sviluppare tali tecnologie da zero, richiede molta precisione ed esperienza da parte dello sviluppatore. Per velocizzare e standardizzare questo processo, vengono ampiamente utilizzati i *framework*, ovvero insiemi organizzati di librerie, convenzioni e codici. *Bootstrap* [4] integra nativamente sistemi a griglia e regole flessibili, che traducono operativamente i principi del *design* responsi-

vo, riorganizzando dinamicamente i blocchi di contenuto in base alla larghezza e all'orientamento dello schermo (*Viewport*) del dispositivo.

Oltre alla gestione del *display*, *Bootstrap* risolve il problema della frammentazione nell'interpretazione degli standard del codice da parte dei vari *browser*, dato che ogni *browser* interpreta gli standard del codice in modo differente. Fogli di stile come *Normalize.css* (integrati in *Bootstrap*) standardizzano i comportamenti di base di elementi grafici, *font* e spaziature. Inoltre, l'architettura integra nativamente gli attributi *Accessible Rich Internet Applications* (WAI-ARIA), fondamentali per descrivere la semantica delle interfacce e garantire la piena compatibilità con gli *screen reader* e le tecnologie assistive.

In questo panorama, *Bootstrap* si posiziona come uno dei *framework frontend* più completi e diffusi al mondo, concepito per consentire una maggiore rapidità di sviluppo.

2.1.4 Interattività tramite JavaScript

Se il HTML è la struttura e il CSS è la presentazione, *JavaScript* è il motore comportamentale del Web. *JavaScript* è un linguaggio di programmazione ad alto livello, multi-paradigma (supportando stili imperativi, funzionali e orientati agli oggetti) a tipizzazione dinamica, la cui esecuzione nei moderni motori di *rendering* avviene tipicamente tramite compilazione *Just-In-Time* (JIT), una tecnica che traduce il codice in linguaggio macchina in tempo reale durante l'esecuzione per ottimizzare dinamicamente le prestazioni. Sebbene nato come linguaggio di *scripting* elementare, si è evoluto nello standard *ECMAScript*. In particolare, il rilascio di ES6 nel 2015 ha colmato il divario con i linguaggi di livello *enterprise*, introducendo il supporto nativo alla programmazione modulare e una sintassi basata su classi per l'ereditarietà, agevolando l'approccio orientato agli oggetti [5].

I programmi *JavaScript* necessitano di un “ambiente *host*” (*host environment*) per interagire con l'esterno che, nel dominio dello sviluppo *frontend*, è il *browser web* stesso. Il *browser* dota *JavaScript* di svariate API (*Application Programming Interface*, interfacce di programmazione esposte all'applicazione). La più importante è il DOM, attraverso la quale il linguaggio può ricercare elementi, mutare l'albero dei nodi, inserire o rimuovere *tag* e modificarne i contenuti.

JavaScript opera seguendo un'architettura guidata dagli eventi e di tipo asincrono. L'applicazione non impegna la CPU in cicli di attesa attiva, ma risponde a stimoli esterni, definiti eventi. Gli eventi possono essere azioni compiute dall'utente, come un clic o la pressione di un tasto, oppure risposte inviate da un *server*. Per rispondere a questi stimoli, gli sviluppatori utilizzano le funzioni di *callback*, ovvero blocchi di istruzioni che vengono attivati dal *browser* solo nel momento in cui si verifica l'evento a cui sono stati associati. Questo meccanismo, orchestrato dall'interazione tra la coda dei messaggi (*callback queue*) e l'*Event Loop*, garantisce la natura non bloccante del linguaggio. Essendo *JavaScript* un *runtime single-threaded*, delegare le operazioni asincrone permette di non occupare il processo (*thread*) principale, preservando la fluidità del *rendering* ed evitando colli di bottiglia nell'esperienza utente [6].

Modelli di Comunicazione Asincrona

La comunicazione asincrona, storicamente definita con l'acronimo AJAX (*Asynchronous JavaScript and XML*), veniva gestita inizialmente tramite *callback*. Oggigiorno per il fenomeno del “*callback hell*” (ovvero l'eccessivo annidamento delle funzioni di ritorno), sono state introdotte le *Promise*, oggetti che rappresentano il risultato futuro e non ancora disponibile di un'operazione. Esse permettono di concatenare in modo lineare le richieste tramite *fetch* e consentono poi di gestire gli errori in modo centralizzato tramite `catch()`, indirizzando i successi attraverso `then`. Per gestire le *Promise* sono stati introdotti le parole chiave `async` e `await`, che permettono di scrivere il codice asincrono, come quello sincrono, sfruttando il medesimo modello non bloccante basato sull'*Event Loop* [6, ch. 13].

La Libreria jQuery

jQuery [7] è una libreria *JavaScript* compatta progettata per ottimizzare la manipolazione del DOM, standardizzare la gestione degli eventi e semplificare l'esecuzione delle richieste asincrone (AJAX).

La gestione delle comunicazioni asincrone è centralizzata all'interno del metodo `jQuery.ajax` (o dell'alias `$.ajax()`), il quale permette di configurare ed effettuare richieste HTTP/HTTPS personalizzate. La libreria dispone inoltre di funzioni per i metodi più comuni, come `jQuery.get()` e `jQuery.post()` per le chiamate

per richiedere e inviare dati al *server* e `jQuery.getJSON()` che carica e valuta direttamente dati codificati in formato JSON. Per la gestione del flusso asincrono, *jQuery* implementa inoltre un proprio modello affine allo standard delle *Promise* attraverso l'oggetto `jQuery.Deferred()`, che fornisce metodi concatenabili come `.done()` per il successo, `.fail()` per gli errori e `.then()` per registrare le *callback* in modo flessibile.

L'adozione di questa architettura facilita il trasferimento strutturato e coerente dei messaggi in JSON verso il *backend*, ponendosi come uno strato di transizione ideale prima che la richiesta venga presa in carico dai sistemi di *routing* lato *server*.

2.2 Sviluppo Lato Server e Linguaggio PHP

2.2.1 Il Modello Architetture Client-Server

Il modello di architettura *Client-Server* è uno dei due principali paradigmi architetture del *web*. In questo modello l'infrastruttura è suddivisa tra due processi comunicanti: un *host* “*always-on*” (il *server*), che possiede un indirizzo IP fisso e noto, e i dispositivi degli utenti (i *client*), che inviano richieste al sistema centrale. Nel contesto dello sviluppo *web*, questa separazione si traduce nella distinzione tra “*frontend*” (il lato *client*) e “*backend*” (il lato *server*) [8, ch. 2.1].

Lo scopo del *server* è gestire richieste provenienti da moltitudini di *host client*. I *client* per contro non comunicano mai direttamente tra di loro nell'architettura *Client-Server*. Essi sono costituiti dai *browser* (*user-agents*) in esecuzione sui dispositivi degli utenti. Le applicazioni *client-side* elaborano l'interfaccia utente utilizzando tecnologie come HTML, CSS e *JavaScript*.

Per la sua stessa natura, tuttavia, il browser costituisce un ambiente intrinsecamente esposto. Il codice viene infatti consegnato in chiaro all'utente, il quale, tramite i normali strumenti per sviluppatori, ha il pieno controllo per leggerlo, analizzarlo e modificarlo a proprio piacimento [2, ch. 2]. Il *server*, quindi, deve agire come garante dell'integrità e della logica del sistema tramite lo *scripting server-side*. Lo sviluppo *backend* si concentra interamente sui *software* e sui *database* in esecuzione sul *server*. Esso deve gestire ogni singola operazione, validazione di *input*, interagire con il database e gestire la memorizzazione dei file. Poiché il *client* è un ambiente esposto alle azioni dell'utente, l'unico modo per assicurare la

validità dei dati è demandare al *server* ogni operazione di verifica [2, ch. 1]. Il *server* agisce come un “filtro”, ogni transazione, sia essa la visualizzazione di un documento riservato o il caricamento di un nuovo file, viene validata dalla logica di *business* residente sul *server*. Se l’utente non possiede i privilegi necessari o se i dati inviati non superano i rigorosi controlli imposti dalle procedure *backend*, la transazione viene bloccata alla radice. In questo ecosistema, il *client* funge esclusivamente da mezzo di visualizzazione e di raccolta degli *input*, mentre il sistema centrale detiene l’autorità assoluta e irrevocabile sulle operazioni e sui registri di *audit*.

2.2.2 Il Protocollo di Comunicazione HTTP

L’*Hypertext Transfer Protocol* (HTTP) [8, ch. 2.2] è il protocollo a livello applicativo fondamentale del *World Wide Web*, impiegato per la trasmissione di dati su Internet. A livello architetturale, l’HTTP rappresenta l’insieme di standard e specifiche gestito dall’*Internet Engineering Task Force* (IETF), l’organizzazione internazionale responsabile della definizione e dello sviluppo dei protocolli di rete. Tali specifiche vengono implementate dai programmi *software* come i *browser* lato *client* e i *web server*, i quali, pur trovandosi in esecuzione su terminali differenti, interagiscono scambiandosi messaggi HTTP. Il protocollo definisce rigorosamente la sintassi, la semantica e la struttura di questi messaggi, nonché le modalità operative con cui le due parti comunicano.

Al fine di garantire la confidenzialità, l’autenticazione e l’integrità dei dati in transito nell’architettura *web*, a questo standard si affianca l’*Hypertext Transfer Protocol Secure* (HTTPS). Esso rappresenta l’estensione sicura del protocollo HTTP. Non ne altera la semantica o la logica dei messaggi del livello applicativo, ma incapsula l’intera comunicazione all’interno di un canale crittografato tramite il protocollo TLS (*Transport Layer Security*), prevenendo l’intercettazione in chiaro o la manomissione delle informazioni da parte di nodi intermedi.

Il ciclo vitale di questa comunicazione è basato su un rigido modello *Request-Response* (Richiesta-Risposta). Il processo ha inizio quando il *client* (tipicamente il *browser* dell’utente) necessita di un’informazione o deve trasmettere dei dati, inviando un messaggio di richiesta HTTP (*HTTP request*) al *server*. Questo messaggio contiene campi di intestazione (*header*) e un metodo (come *GET* per ri-

chiedere dati, POST per inviare, DELETE per rimuoverli). Il *server* è costantemente in ascolto su una porta specifica, la porta 80 per le connessioni HTTP standard non crittografate e la porta 443 per il traffico protetto HTTPS. Una volta che il *server* ha ricevuto questa richiesta, recupera, assembla e invia le informazioni sotto forma di messaggio di risposta HTTP (*HTTP response*) il più rapidamente possibile. Questa risposta contiene un codice di stato (che indica il successo o il fallimento dell'operazione, come "200 OK" o "404 Not Found") e nel corpo (*body*) del messaggio, i dati o la pagina elaborata da restituire all'utente.

L'HTTP è un protocollo *stateless* (senza stato), quindi ogni transazione è indipendente dalle precedenti. Il *server* elabora ciascuna richiesta e invia la relativa risposta al *client* senza conservare memoria delle interazioni già avvenute. Ogni ciclo *Request-Response* costituisce quindi un evento autonomo e isolato. Ad esempio, se un utente richiede lo stesso documento due volte a distanza di pochi secondi, il *server* elaborerà entrambe le richieste come nuove operazioni, senza alcuna conoscenza dell'azione precedente.

Per introdurre un meccanismo di persistenza delle informazioni (necessario, ad esempio, per mantenere una sessione di *login* attiva), vengono utilizzati strumenti aggiuntivi rispetto al protocollo HTTP. Tra questi vi sono i *cookie*, piccoli *file* di dati che il *server* invia al *browser* e che vengono memorizzati sul dispositivo dell'utente. A ogni richiesta successiva, il *browser* può restituire tali informazioni al *server*, consentendo di riconoscere l'utente e mantenere uno stato applicativo. In questo modo è possibile gestire sessioni utente, tracciare le attività effettuate e mantenere registri di *audit* sicuri per ogni transazione eseguita.

L'affidabilità della trasmissione HTTP è delegata a livello di trasporto al protocollo TCP (*Transmission Control Protocol*). Mentre l'HTTPS gestisce la sicurezza crittografica contro le manomissioni volontarie, il TCP risolve le instabilità strutturali della rete e fornisce un servizio orientato alla connessione che previene perdite, duplicazioni o corruzioni accidentali dei dati, assicurandone il corretto riassettaggio. L'integrazione di questi due livelli dota il sistema di un canale di comunicazione che risulta meccanicamente affidabile e crittograficamente sicuro.

Posizione	Linguaggio di Programmazione	Percentuale (%)
1	PHP	71,4%
2	Ruby	6,7%
3	JavaScript (Node.js)	6,2%
4	Java	5,4%
5	Scala	5,0%
6	ASP.NET	4,4%
7	Static files	2,1%
8	Python	1,2%
9	ColdFusion	0,2%
10	Perl	0,1%

Tabella 2.1: Utilizzo dei linguaggi di programmazione lato server [9].

2.2.3 Evoluzione e Paradigmi del Linguaggio PHP

Il linguaggio PHP, acronimo ricorsivo per *Hypertext Preprocessor* è uno dei linguaggi di *scripting server-side* di riferimento per lo sviluppo *web*. Come mostrato nella Tabella 2.1, PHP è la tecnologia più utilizzata nel 71,4% dei casi, nei sistemi *web*, che dispongono di un *back-end*. A titolo di confronto, il suo concorrente più vicino (Ruby) detiene solo il 6,7% del mercato. Questa diffusione capillare si traduce in un ecosistema maturo e ampiamente supportato, esistono moltissime librerie testate (tramite strumenti come *Composer*), documentazioni, libri, *tutorial*, *framework* basati su questo linguaggio e alla garanzia di un supporto continuo a lungo termine.

La scelta di PHP come linguaggio *back-end* garantisce l'utilizzo di una tecnologia ampiamente collaudata su scala globale e supportata da un'infrastruttura *server* massiva. Il linguaggio inoltre offre un'integrazione nativa con principali sistemi di gestione di database relazionali (RDBMS).

L'attuale livello di maturità architetturale è il risultato di una profonda e stratificata evoluzione tecnica. PHP nasce e si sviluppa con il preciso e unico intento di essere un linguaggio per lo *scripting* lato *server*. Viene concepito originariamente nel 1994 da Rasmus Lerdorf come un semplice insieme di file binari CGI (*Common Gateway Interface*) scritti in linguaggio C e denominati *Personal Home*

Page Tools. Il primo punto di svolta avvenne con il rilascio della versione PHP 3.0 nel 1998, derivata da una riscrittura completa del *parser* interno ad opera di Andi Gutmans e Zeev Suraski, dove per la prima volta introdussero il supporto, ancora embrionale, alla programmazione orientata agli oggetti (OOP) e permise l'integrazione di numerosi database in modo nativo. Lo sviluppo delle prestazioni e della modularità è proseguito con l'introduzione dello *Zend Engine* in PHP 4.0, ottimizzando drasticamente le prestazioni del codice e la modularità per applicazioni *enterprise*, e si è consolidato con PHP 5, che ha offerto un modello a oggetti completamente rinnovato e molto più robusto. Le iterazioni più recenti, segnatamente PHP 7 e PHP 8, hanno ulteriormente consolidato il paradigma OOP introducendo l'uso di tipizzazioni strette (*union types*, *named arguments*), la compilazione *Just-In-Time* (JIT) e una gestione delle risorse di memoria nettamente ottimizzata [10].

Un fattore chiave in questa maturazione è l'evoluzione verso il *Gradual Typing*. Storicamente nato come linguaggio a tipizzazione debole e dinamica, PHP ha implementato un sistema di controllo dei tipi progressivo e rigido. L'introduzione degli *Union* e *Intersection Types*, combinata con la direttiva di tipizzazione rigida, permette una precisa definizione dei contratti del codice. Ciò agevola gli strumenti di analisi statica nel rilevare anomalie e incongruenze già in fase di sviluppo [11].

Il linguaggio di programmazione PHP si è strutturato come un ecosistema multiparadigma. I paradigmi principali supportati e comunemente utilizzati nello sviluppo moderno sono tre [11]:

- **Paradigma Orientato agli Oggetti (OOP):** Rappresenta il paradigma dominante utilizzato dai *framework* moderni come Laravel. Il linguaggio supporta l'incapsulamento rigoroso tramite modificatori di visibilità, l'ereditarietà singola, le interfacce, le classi astratte e il polimorfismo.
- **Paradigma Procedurale:** Costituisce la base storica del linguaggio e l'architettura nativa di molte delle sue funzioni principali. Si basa su una struttura lineare di funzioni e costrutti di controllo imperativi, questo modello è particolarmente indicato per l'implementazione di *script* d'automazione e componenti *software* a bassa complessità.
- **Supporto al Paradigma Funzionale:** Sebbene non sia un linguaggio funzionale nativo, PHP implementa funzioni di prima classe, funzioni ano-

nime (*Closure*) e funzioni freccia (*Arrow Functions*). Inoltre, sono stati implementati costrutti nativi di ordine superiore come `array_map()` e `array_filter()`, che consentono l'elaborazione dei dati secondo logiche dichiarative e non imperative.

2.2.4 Gestione delle Dipendenze tramite Composer

Composer [12] è uno strumento di gestione delle dipendenze in *PHP*. A differenza dei tradizionali gestori di pacchetti, come *Yum* o *Apt*, *composer* gestisce i pacchetti/librerie su base per progetto e le installa in una *directory* locale (solitamente denominata *vendor*). Tale architettura, concettualmente ispirata a strumenti quali *npm* o *bundler*, permette di dichiarare, gestire e installare le librerie software esterne (pacchetti) richieste da un progetto, risolvendo automaticamente le relazioni di interdipendenza tra di esse.

Composer è formato da tre elementi fondamentali:

- **composer.json**: *File* di configurazione in formato *json* al nucleo del progetto. Contiene i metadati dell'applicazione e l'elenco delle dipendenze dirette per lo sviluppo (*require-dev*) e per la produzione (*require*).
- **composer.lock**: *File* generato automaticamente dopo la prima installazione o l'aggiornamento dei pacchetti. Registra le versioni esatte di ciascun pacchetto, così da poter garantire *build* riproducibili e deterministiche.
- **Packagist**: Il *repository* pubblico principale e predefinito di *Composer*. Aggrega i pacchetti *open source PHP* disponibili globalmente e viene interrogato da *Composer* per localizzare i pacchetti da scaricare.

L'algoritmo di risoluzione individua le dipendenze corrette interrogando *Packagist* per raccogliere i metadati dei pacchetti richiesti. Successivamente, analizza i *tag* e i rami dei sistemi di controllo di versione (VCS) per valutare i vincoli di versione e calcolare la combinazione di librerie compatibile con le richieste del software. Infine, il processo termina definendo le versioni esatte di ciascun pacchetto che verranno poi cristallizzate all'interno del *file composer.lock*.

Composer dispone inoltre di un *autoloader*, che è un meccanismo che automatizza il caricamento dei *file PHP* all'interno del progetto, eliminando la necessità di inserire manualmente istruzioni come *require* o *include* in ogni *script*. Adottando standard rigorosi di interoperabilità come PSR-4, il sistema di *autoloading* mappa in modo trasparente i *namespace* (spazi dei nomi) logici delle classi sui percorsi fisici del *filesystem*. L'inclusione dell'*Autoloader* all'interno del progetto consente il caricamento e l'utilizzo dinamico delle classi a *runtime*, nel momento esatto in cui vengono istanziate. Questo processo si chiama *Lazy loading*, quindi quando il programma cerca di istanziare o utilizzare una classe non caricata in memoria, l'*Autoloader* intercetta la richiesta, controlla la mappa precedentemente generata e include dinamicamente il *file* richiesto.

2.3 Architettura del Framework Laravel

Laravel è un *framework full-stack open source* scritto in PHP. *Laravel* è al momento, secondo le indagini di mercato condotte annualmente da *JetBrains* (all'interno del *report State of Developer Ecosystem*), il *leader* assoluto fra i *framework* PHP, scelto da oltre il 64% degli sviluppatori PHP. [13]

Il suo obiettivo primario è garantire una “*developer experience*” eccellente, mettendo a disposizione strumenti potenti e raffinati per gestire la complessità delle applicazioni moderne.

Inoltre, *Laravel* si definisce un *framework* progressivo, scalabile e *community driven*. Progressivo, dato che è stato ideato per crescere insieme allo sviluppatore *web* ed alle esigenze del progetto, risultando accessibile sia per sviluppatori *junior*, offrendo però strumenti robusti per lo sviluppo di applicazioni di livello *enterprise*. Scalabile, invece grazie alle caratteristiche di PHP e al supporto integrato per sistemi di *cache* distribuita e veloce come *Redis*, che facilitano lo *scaling* orizzontale ed infine *community driven*, dato che si basa sul contributo di migliaia di sviluppatori in tutto il mondo. [14]

L'architettura del *framework* è progettata per garantire una rigorosa separazione delle responsabilità (*Separation of Concerns*). Questo obiettivo viene conseguito tramite quattro pilastri metodologici: la gestione del *routing* per l'instradamento delle richieste HTTP, l'adozione del *pattern* MVC per la strutturazione del codi-

ce, la gestione delle dipendenze tramite il meccanismo di *Dependency Injection*, e infine, l'astrazione dello strato di persistenza dei dati.

2.3.1 Il Pattern Architetturale MVC

Il *framework Laravel* adotta nativamente il *pattern* architetturale *Model-View-Controller* (MVC) per separare in modo netto la logica di presentazione dalla logica di *business* e di persistenza dei dati.

View

[15] La vista [16] è completamente isolata dalla logica di *business* e non possiede logica decisionale. Non sa da dove provengono i dati, né se siano stati elaborati. Il suo unico compito è quello di esporre le informazioni inoltrate. La *View* viene gestita tramite il motore di *template Blade* che, a partire da file con estensione `.blade.php`, genera dinamicamente il codice HTML. Questi file vengono salvati tipicamente nella *directory* `resources/views`. *Blade* è un motore *template* molto leggero, accetta codice PHP puro all'interno delle viste e compila le viste in codice PHP che viene memorizzato nella *cache* finché non viene modificato, il che significa che *Blade* non aggiunge praticamente alcun sovraccarico (*overhead*) all'applicazione.

Le viste sono organizzate in modo gerarchico: è possibile definire scheletri strutturali globali (*layout*) che descrivono parti fisse dell'interfaccia, come *header* o *footer*, che possono venir riutilizzati in ogni pagina. Allo stesso modo si possono creare componenti più piccoli, come bottoni specifici, che possono essere incapsulati e riutilizzati ovunque.

La sintassi standard di *Blade* (`{{ $variabile }}`) sanifica automaticamente l'output tramite la funzione `htmlspecialchars()`, bloccando all'origine attacchi *Cross-Site-Scripting* (XSS) nel corpo della pagina HTML.

Model

Il Modello [17] è incaricato di definire la struttura dei dati e di governare l'interazione concettuale con il database. Esso funziona tramite un'astrazione orientata agli oggetti, in cui ogni entità del sistema corrisponde ad una classe, che incapsula

le proprietà del dato e le relative regole di *business*. Vengono anche definite le relazioni con altri oggetti (per esempio, una struttura può contenere più stanze).

Il Modello, quindi, fornisce un'interfaccia ad alto livello per interrogare, modificare, o ricevere informazioni dal database attraverso metodi e oggetti *software*, senza dover scrivere manualmente istruzioni nel linguaggio nativo del database.

Il modello inoltre garantisce la coerenza logica del dato applicando le regole stabilite. Gestisce la formattazione automatica dei dati (es. *cast* da stringa a data), la protezione dei campi sensibili e la definizione di attributi calcolati.

Controller

Il Controllore [15] è il coordinatore fra i vari strati. Invece di definire tutta la logica di gestione all'interno dei file di instradamento (*routing*), *Laravel* incoraggia fortemente a raggruppare i comportamenti correlati all'interno di classi Controllore dedicate. Esso ha il compito di raggruppare e organizzare la logica necessaria per gestire le richieste HTTP in ingresso. Riceve la richiesta, la analizza e poi stabilisce quale risposta restituire; in un'architettura *full-stack* il Controllore restituirà tipicamente un'istanza di una Vista *Blade*, dopo aver interrogato uno o più modelli per ottenere le relative informazioni.

I Controllori sono di solito basati su un'architettura delle risorse (RESTful), dove esistono metodi standard per un insieme standard di azioni (come visualizzare un singolo elemento, mostrare un elenco, eliminare un elemento, modificarlo, *etc.*). Questo ha lo scopo di standardizzare i controllori, così da rendere la logica interna pulita e prevedibile.

In sintesi il ciclo MVC è il seguente (mostrato in Figura 2.2):

1. Il controllore riceve la richiesta dall'utente e determina l'azione da eseguire.
2. Il Controllore interroga uno o più Modelli per ottenere o modificare le informazioni necessarie.
3. Il Modello esegue la logica d'affari e la persistenza dei dati, restituendo i risultati al Controllore.
4. Infine, il Controllore assembla queste informazioni e le passa ad una Vista, che genera il risultato finale da presentare all'utente.

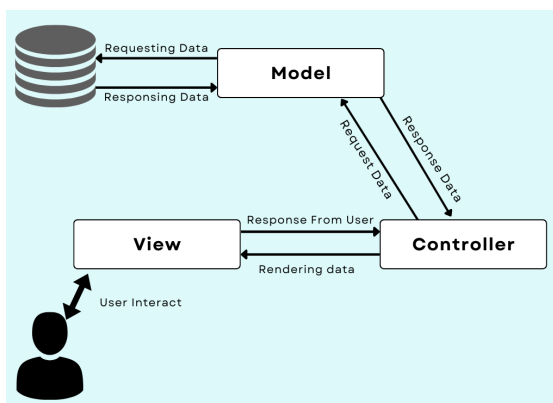


Figura 2.2: Rappresentazione del flusso di esecuzione nel pattern MVC [18].

2.3.2 Il Ciclo di Vita della Richiesta

Il ciclo di vita della richiesta [19] (*Request Lifecycle*) è il processo architetturale e temporale mediante il quale il *framework Laravel* intercetta una richiesta HTTP inviata da un *client*, inizializza l'ambiente di esecuzione, effettua i controlli di sicurezza e indirizza la richiesta verso la logica applicativa corretta, gestendo infine la restituzione della risposta.

Fase di Inizializzazione (Bootstrap)

Il ciclo di vita della richiesta ha inizio nel momento in cui il *server web* riceve una richiesta HTTP indirizzata all'applicazione, che convoglia tutte le richieste verso il file `public/index.php`. Questo file è il punto di partenza per caricare il resto del *framework*. Ha il compito di caricare la definizione dell'*autoloader* generata dal gestore di dipendenze *Composer* per poi istanziare la prima istanza dell'applicazione e del *Service Container* tramite `bootstrap/app.php`.

Infine, l'applicazione assegna la gestione delle richieste al *Kernel* HTTP. Il *Kernel* esegue sempre una serie di *bootstrapper* prima che la richiesta venga effettivamente gestita, i quali configurano la gestione degli errori, il *logging*, rilevano l'ambiente dell'applicazione e svolgono altre attività interne.

In questa fase vengono anche caricati i *Service Providers* [20], questi sono responsabili dell'inizializzazione (*bootstrapping*) di tutti i vari componenti del *framework*, come il database, le code, la validazione e il *routing*. *Laravel* itera

attraverso l'elenco dei *provider*, li istanzia, chiama il loro metodo `register` e successivamente, una volta tutti registrati, invoca il metodo `boot`.

Filtrazione tramite Middleware

Il *Kernel* HTTP è anche responsabile del passaggio della richiesta attraverso lo *stack* di *middleware* [21] dell'applicazione. Ogni *middleware* è una classe indipendente dotata di un metodo standard `handle($request, Closure $next)` e costituisce un meccanismo per ispezionare, filtrare ed eventualmente respingere le richieste HTTP in entrata ed uscita. Il passaggio della richiesta allo strato successivo è unicamente possibile invocando `$next($request)`.

I *middleware* sono divisi in due categorie:

I *middleware* globali, che rappresentano lo strato più esterno della *pipeline* e vengono eseguiti su ogni richiesta HTTP in ingresso; il loro compito è validare lo stato globale dell'infrastruttura. I *middleware* di rotta, che sono filtri eseguiti solo dopo che il *framework* ha analizzato l'URI della richiesta e ha determinato la rotta di destinazione. A questa categoria appartengono *middleware* come *Authenticate*, che verifica che l'utente sia autorizzato, o *VerifyCsrfToken*, che implementa la difesa contro attacchi *Cross-Site Request Forgery* (CSRF).

L'Instradamento tramite Routing

Dopo che la richiesta supera i *middleware* globali, viene consegnata al *router* [22] per l'instradamento. Il *router* di *Laravel* agisce come un mappatore deterministico: analizza i metadati della richiesta in entrata, isolando specificamente l'URI (la stringa dell'indirizzo, come `/profilo`) e il metodo HTTP associato (GET, POST, PUT, DELETE).

Il *router* confronta questa richiesta con l'elenco delle rotte definite all'interno della cartella `/routes`. Queste rotte accettano un URI preciso insieme ad un controllore o una *closure*.

Il *router* esamina l'URI in ingresso e lo fa corrispondere a una rotta definita, eseguendo contestualmente eventuali *middleware* specifici assegnati a quella rotta o a quel gruppo di rotte. Se la richiesta supera con successo tutti i *middleware* assegnati, il *router* effettua l'`_handover_`, che individua la classe controllore e invoca il metodo associato oppure esegue la *closure* della rotta.

L'esecuzione della rotta o del *controller* genera infine una risposta HTTP. Questa risposta compie un viaggio a ritroso passando nuovamente attraverso la catena di *middleware* della rotta, dando all'applicazione la possibilità di modificare o esaminare la risposta in uscita. Infine, il metodo `handle` del *Kernel* HTTP restituisce l'oggetto risposta, e il contenuto viene inviato al *browser* dell'utente, concludendo così il ciclo di vita della richiesta.

2.3.3 Service Container

Il *Service Container* [23] si occupa di gestire le dipendenze delle classi e di eseguire le iniezioni delle dipendenze (*Dependency Injection*, DI).

La DI implementa in modo operativo il paradigma dell'Inversione del Controllo (*Inversion of Control*, IoC), che consiste nel delegare la gestione delle dipendenze a un componente esterno, come il *framework*, invece che alle singole classi. In questo modo la DI rappresenta la concretizzazione di questo principio: le dipendenze non vengono più create internamente dal componente che le richiede, ma vengono “iniettate” dall'esterno al momento della sua attivazione.

Il *Service Container* di *Laravel* automatizza la *Dependency Injection* tramite l'*auto-wiring* (Risoluzione Automatica). Sfruttando la *Reflection* API di PHP, il *container* analizza il costruttore della classe per identificare il *type-hinting*, ovvero le dichiarazioni esplicite di classi o interfacce sui parametri del metodo. Una volta determinato il tipo richiesto, il *container* risolve ricorsivamente la dipendenza, istanziando e configurando completamente l'istanza della classe principale, eliminando così la necessità di mappature o configurazioni manuali.

Il *pattern* del *Service Layer* è un esempio concreto che illustra l'efficacia della *Dependency Injection* (DI) e del *Service Container* in *Laravel*. L'inclusione della logica di *business* complessa direttamente nei *Controller* porta a un codice rigido, fortemente accoppiato e difficile da testare.

La soluzione consiste nell'estrarre queste operazioni in una classe separata, chiamata Servizio, e “iniettarla” nel *Controller* tramite il costruttore. In questo modo, il *Controller* si occupa solo della gestione del traffico HTTP, delegando l'elaborazione dei dati al Servizio.

2.4 Persistenza dei Dati e Database Relazionali

I sistemi di basi di dati (DBMS) rappresentano l'infrastruttura standard per l'archiviazione strutturata, la persistenza e il recupero delle informazioni nei moderni sistemi *software*.

Il paradigma principale per l'organizzazione logica dei dati è il modello relazionale, introdotto formalmente da Edgar F. Codd nel 1970. Questo modello, a differenza dei modelli gerarchici o reticolari precedenti, si basa sulla teoria matematica degli insiemi e sull'algebra relazionale. In questo modello, le informazioni sono organizzate in tabelle, chiamate relazioni, composte da righe, o tuple, che rappresentano singole istanze di un'entità, e colonne, o attributi, che descrivono le proprietà di quell'entità. La solidità matematica del modello consente di interrogare e manipolare i dati in modo dichiarativo, separando la logica applicativa dalle modalità fisiche di memorizzazione delle informazioni. Tale separazione permette una maggiore indipendenza tra i processi di sviluppo *software* e le tecniche di gestione dello spazio di archiviazione.

2.4.1 Sistemi di Gestione di Database

La gestione del modello relazionale è affidata ai sistemi di gestione di basi di dati (DBMS). I DBMS sono sistemi *software* avanzati progettati per gestire collezioni di dati, garantendo che tali dati siano condivisi, persistenti e mantenuti con elevati standard di affidabilità e *privacy*. Queste collezioni di dati possono essere di dimensioni molto grandi, rendendo necessaria la gestione dell'allocazione dei dati in memorie secondarie. Il DBMS garantisce la condivisione tra molteplici utenti e applicazioni, eliminando la ridondanza strutturale e prevenendo il rischio di inconsistenze logiche derivanti dalla replicazione non controllata. Questo viene implementato tramite rigorosi meccanismi di controllo della concorrenza da parte dei DBMS, i quali governano gli accessi simultanei garantendo stabilità transazionale anche su volumi di milioni di operazioni al secondo. Infine, il DBMS garantisce l'affidabilità dei dati, che possono essere ripristinati pure in caso di guasti *hardware* o *software* (*backup* e *recovery*), e di sicurezza degli accessi tramite meccanismi di autorizzazione che abilitano gli utenti solo a specifiche azioni.

L'architettura interna di un DBMS relazionale (RDBMS) si compone di vari sottosistemi: il *Query Processor*, che analizza, ottimizza ed esegue le interrogazioni formulate nel linguaggio standard SQL (*Structured Query Language*); lo *Storage Manager*, che interfaccia il motore logico con il *file system* del sistema operativo per l'allocazione dello spazio su disco; e il *Transaction Manager*, responsabile della sicurezza e della concorrenza. L'impiego di un DBMS solleva lo sviluppatore dall'onere di gestire l'accesso a basso livello ai *file*, la frammentazione dei dischi e le collisioni derivanti dall'accesso simultaneo ai dati da parte di molteplici processi.

2.4.2 Eloquent ORM

L'*Object-Relational Mapping* (ORM) è un paradigma architetturale ideato per colmare l'incongruenza strutturale e logica che esiste fra la programmazione orientata agli oggetti, come PHP, e i RDBMS. Eloquent è la soluzione ORM integrata in Laravel, sviluppata secondo il *design pattern Active Record*. In conformità a questo *pattern*, ogni tabella del database viene associata a una classe *Model*, le cui istanze incapsulano simultaneamente i dati della singola tupla e la logica necessaria per l'accesso e la manipolazione del *record* corrispondente.

L'architettura di Eloquent si basa sul principio di convenzione rispetto alla configurazione: se lo sviluppatore non specifica esplicitamente il nome della tabella di riferimento, il sistema lo deduce automaticamente convertendo il nome della classe del modello dal formato singolare al plurale in *snake_case*. Allo stesso modo, Eloquent assume sempre la presenza di una chiave primaria intera e autoincrementante chiamata `id` e di colonne *timestamp*, `created_at` e `updated_at`, che l'ORM gestisce automaticamente durante le operazioni di inserimento e aggiornamento.

Inoltre, Eloquent astrae le interazioni tra le tabelle del database implementandole come metodi all'interno delle classi modello. Secondo la documentazione tecnica, il sistema gestisce automaticamente le relazioni standard, come quelle uno-a-uno (*hasOne*) e uno-a-molti (*hasMany*), oltre alle associazioni molti-a-molti, manipolando le tabelle intermedie senza richiedere l'intervento diretto dello sviluppatore.

2.4.3 Versionamento del Database: Migrations

Accanto al sistema ORM Eloquent, Laravel mette a disposizione un meccanismo denominato *Migrations*, progettato per gestire in modo strutturato e versionato l'evoluzione dello schema del database. Le *migrations* possono essere considerate come un sistema di controllo delle versioni per la struttura della base di dati, analogo ai sistemi di versionamento del codice sorgente, ma applicato alle tabelle, ai vincoli e agli attributi del database.

Tradizionalmente, le modifiche allo schema vengono effettuate mediante l'esecuzione diretta di istruzioni SQL sul database. Tale approccio può risultare problematico nei progetti collaborativi o in sistemi soggetti a frequenti evoluzioni, poiché rende complessa la sincronizzazione delle modifiche tra ambienti differenti e aumenta il rischio di inconsistenze strutturali. Le *migrations* affrontano questo problema rappresentando ogni modifica come una classe PHP che descrive esplicitamente le operazioni da applicare.

Ogni *migration* implementa generalmente due metodi principali: `up()` e `down()`. Il metodo `up()` definisce le modifiche da applicare allo schema, come la creazione di nuove tabelle, l'aggiunta di colonne o la definizione di relazioni; il metodo `down()` descrive invece le operazioni inverse necessarie per annullare tali modifiche. Questo approccio consente di eseguire procedure di *rollback*, riportando il database a uno stato precedente nel caso di errori o necessità di ripristino.

Laravel mette inoltre a disposizione uno strato di astrazione chiamato *Schema Builder*, che consente di definire le strutture del database attraverso un'interfaccia orientata agli oggetti, riducendo la dipendenza da specifiche implementazioni SQL. In questo modo, lo sviluppatore può descrivere lo schema in maniera dichiarativa, mantenendo una maggiore portabilità tra differenti sistemi di gestione di database supportati dal *framework*.

A completamento del versionamento strutturale, Laravel integra il meccanismo dei *Seeders*. Mentre le *migrations* operano a livello di definizione dello schema, i *seeders* gestiscono il popolamento automatizzato delle tabelle mediante *dataset* iniziali o di esempio. Tale meccanismo separa logicamente la struttura del database dai dati in esso contenuti, permettendo di automatizzare la configurazione dell'ambiente applicativo e garantendo la riproducibilità dello stato iniziale del sistema durante le fasi di sviluppo, *testing* e distribuzione. L'impiego delle *migrations* fa-

vorisce quindi la tracciabilità delle modifiche, la riproducibilità degli ambienti di sviluppo e produzione e una gestione più controllata dell'evoluzione del database durante l'intero ciclo di vita del *software*.

Progetto ENVASS

3.1 Ente di riferimento: CIAS

Il progetto è stato commissionato dal CIAS [24] (Centro ricerche Inquinamento Fisico Chimico Microbiologico Ambienti Alta Sterilità) dell'Università degli Studi di Ferrara. Il Centro è stato fondato nel 2005 come struttura di ricerca interna del Dipartimento di Architettura. Nel 2013, è stato trasformato in Centro Interdipartimentale dell'Ateneo ed attualmente coinvolge il Dipartimento di Architettura e il Dipartimento di Scienze dell'Ambiente e della Prevenzione, che dal 2024 ne costituisce il dipartimento di afferenza.

Il CIAS integra ricerca scientifica, attività diagnostica e ingegneria impiantistica per la gestione del controllo della contaminazione e l'efficienza energetica in ambienti confinati complessi. Le sue attività si articolano in quattro principali macroaree:

1. *Pollution & Health*: studio e soluzioni per inquinanti fisici, chimici e microbiologici, presenti nell'aria e sul terreno, e i loro effetti sulla salute delle persone e degli animali. Include la validazione sperimentale di protocolli di sanificazione clinica o industriale (aree GMP) e lo sviluppo di sistemi *bio-based* per il contrasto delle Infezioni Correlate all'Assistenza (ICA).

2. Ambiente e Energia: analisi ambientali per ottimizzare il *comfort*, la qualità dell'aria e l'efficienza energetica, con un *focus* sui requisiti tecnici e strutturali di ambienti ad alta sterilità come blocchi operatori e laboratori.
3. *Heritage*: analisi dei fenomeni di degrado biologico, fisico e chimico di beni di interesse storico-artistico, con l'obiettivo di preservarli in modo efficace. Questo include la progettazione di impianti adeguati per la protezione degli spazi espositivi e museali.
4. *Safety & Training*: ricerca applicata nel settore della sicurezza sul lavoro e della prevenzione incendi. Accoppiata ad attività di formazione specialistica e aggiornamento professionale. Le attività riguardano procedure di biosicurezza e tecniche di analisi microbiologica.

Le attività di campionamento e le analisi di laboratorio sono condotte in conformità a metodi ufficiali accreditati e seguono rigorosi protocolli standardizzati. Le procedure operative sono suddivise in due fasi sequenziali e interdipendenti. La prima riguarda i metodi di campionamento, che stabiliscono le operazioni da svolgere sul campo e la strumentazione necessaria per il prelievo rappresentativo della matrice ambientale, come stabilito dalla norma UNI EN 13098 per la pianificazione del campionamento di *bioaerosol*. La seconda è rappresentata dai metodi di prova, che stabiliscono i protocolli analitici e microbiologici di laboratorio per la determinazione e la quantificazione della contaminazione, in conformità con *standard* come la norma UNI EN ISO 4833 per la conta dei microrganismi vivi. In laboratorio, queste fasi costituiscono un ciclo operativo unico, formalmente convalidato congiuntamente dai due *standard* (UNI EN 13098 + UNI EN ISO 4833). La conformità delle procedure e la competenza tecnica della struttura sono attestate dall'accreditamento di Accredia (Ente Italiano di Accreditamento), che ne garantisce l'affidabilità tramite verifiche ispettive e controlli periodici.

3.2 Descrizione del progetto

Il progetto ENVASS nasce e si sviluppa all'interno del laboratorio CIAS per rispondere all'esigenza di creare un sistema gestionale centralizzato in grado di archiviare, tracciare e gestire tutte le informazioni relative ai campioni analizzati.

Nella sua prima versione, il sistema era un'interfaccia *web* statica, usata principalmente per catalogare manualmente i singoli campioni analizzati e le relative foto. Erano disponibili anche funzionalità di estrazione dati basati su filtri di ricerca le cui variabili chiave erano il progetto di riferimento, la struttura ospitante e le date di rilevazione.

In seguito, per rispondere alle crescenti richieste di digitalizzazione e ai severi requisiti di conformità e tracciabilità imposti dall'ente nazionale di accreditamento ACCREDIA, è stato necessario aggiornare la piattaforma con un'architettura più moderna e integrata, in grado di estendere il controllo informatico a tutte le fasi della filiera operativa e procedurale del laboratorio. Per realizzare questa transizione tecnologica, il CIAS ha affidato al laboratorio ACLAI la progettazione e lo sviluppo di una versione avanzata del sistema preesistente. L'analisi del flusso di lavoro ha portato alla mappatura dei processi principali e allo sviluppo di un'architettura *software* suddivisa nei macro-blocchi, che andremo ad analizzare di seguito.

3.3 Anagrafica

L'anagrafica svolge il ruolo di registro centrale delle entità strutturali, dei soggetti e delle attività di laboratorio.

Innanzitutto, il modulo gestisce i profili dei committenti, ovvero i clienti esterni o gli enti terzi che richiedono le prestazioni del CIAS. Per ciascun committente, il sistema archivia le informazioni di contatto, che includono recapiti telefonici, indirizzi di posta elettronica e sedi fisiche. Questo garantisce la tracciabilità amministrativa e la corretta associazione delle commesse.

L'anagrafica rappresenta i siti attraverso un modello gerarchico articolato su tre livelli. Le strutture identificano le entità principali, come complessi edilizi, presidi ospedalieri o siti industriali. Le partizioni definiscono le principali suddivisioni logiche o fisiche. Infine, le aree di partizione introducono un ulteriore livello di dettaglio, individuando specifici elementi spaziali all'interno delle partizioni, quali singoli edifici e relativi piani.

Le attività, invece, sono i progetti assegnati e gestiti dal laboratorio. Ognuna è caratterizzata da attributi che comprendono una tipologia specifica che ne deter-

mina la natura e la finalità (ad esempio, progetto di ricerca, monitoraggio, ecc.) e uno stato operativo.

Infine, nel blocco degli utenti viene censito il personale autorizzato a interagire con il sistema *software* e a ciascun profilo viene assegnata una specifica matrice di diritti e permessi operativi. Questa categorizzazione è fondamentale per regolamentare in modo gerarchico i livelli di accesso alle funzionalità di inserimento, modifica o consultazione dei dati, garantendo la sicurezza informatica e l'integrità delle informazioni in conformità con i requisiti ispettivi.

3.4 Liste

Il macro-blocco delle Liste costituisce il dizionario tecnico e il catalogo centralizzato del sistema ENVASS, raccogliendo i parametri chimici, biologici, strumentali e metodologici essenziali per la descrizione e la standardizzazione del progetto di campionamento e delle analisi. Il modulo offre un archivio di voci predefinite e codificate che riducono i tempi di inserimento manuale e garantiscono l'uniformità dei dati secondo le direttive di controllo della qualità.

Il sistema definisce la configurazione spaziale e operativa dei prelievi tramite i punti di campionamento, che vengono associati a specifiche matrici e a dettagli descrittivi dell'area di indagine.

Una parte fondamentale del modulo è dedicata al censimento degli strumenti di campionamento. Per ogni apparecchiatura vengono registrati il fornitore, l'utilizzo previsto, le date di acquisto e di attivazione, lo stato operativo e i criteri di accettazione. Per garantire la rintracciabilità metrologica, il *software* traccia lo stato di approvazione, i dati di taratura, la manutenzione e lo storico delle revisioni periodiche, annotando la data, il tecnico esecutore, il responsabile della verifica e le eventuali note.

La standardizzazione delle procedure si basa sulla distinzione e sull'integrazione di due componenti fondamentali: i metodi di campionamento e i metodi di prova. I metodi di campionamento definiscono le operazioni di prelievo sul campo e includono i dati relativi alla matrice, all'ambiente e al parametro, con la loro specifica denominazione. I metodi di prova, invece, governano la fase successiva di analisi in laboratorio, memorizzando le informazioni sulla matrice, sull'ambiente, sul parametro biologico da ricercare, sulla temperatura e sul tempo di incubazio-

ne necessari, nonché una descrizione testuale della procedura e il tipo di verbale applicabile. Inoltre, il macroblocco contiene la gestione dei lotti dei materiali di consumo, in cui vengono tracciati il terreno di coltura, il tipo di supporto, il fornitore e la data di scadenza.

Il modulo contiene tabelle per piastre, microrganismi e antibiotici utilizzati per i *test* di suscettività e resistenza. La sezione include anche l'elenco delle stanze e le linee guida, che contengono i riferimenti normativi e i limiti di accettabilità associati ai controlli del laboratorio. Infine, sono incluse tabelle secondarie meno rilevanti ma indispensabili alla funzionalità del sistema.

3.5 Verbalì

Nella parte iniziale del verbale sono riportati i dati anagrafici necessari per raggiungere il luogo del prelievo e un riepilogo generale del monitoraggio da effettuare. Ogni verbale viene generato a partire da una "Campagna di campionamento". Questa definisce a monte il committente, l'attività e la struttura di riferimento. Il rapporto tra queste due entità è di tipo uno-a-molti: una singola campagna può contenere più verbalì, mentre un verbale appartiene rigorosamente a una sola campagna.

Il documento viene creato unendo i dati della campagna, la specifica partizione, l'area di partizione e la tipologia di verbale, che può essere interno o esterno. Questa distinzione stabilisce chi esegue fisicamente il prelievo: se il verbale è interno, il campionamento viene svolto dai tecnici del CIAS; se invece è esterno, viene eseguito da un ente terzo che consegna successivamente i campioni al laboratorio.

Indipendentemente dalla tipologia scelta, le informazioni contenute nel verbale si suddividono sempre in tre categorie principali. La prima riguarda le stanze, le modalità e i punti di campionamento, definendo con precisione dove, come e cosa viene analizzato.

La seconda categoria include gli strumenti utilizzati e la planimetria del luogo. La terza categoria è l'accettazione, che viene compilata al rientro dal campionamento per i verbalì interni o alla ricezione per quelli esterni, per registrare i dettagli del prelievo e del trasporto appena effettuati.

Una volta completata la fase di accettazione, il sistema genera automaticamente un documento in formato PDF che funge da attestazione ufficiale e immutabile.

dei dati inseriti. Una volta verificata la correttezza delle informazioni, il sistema genera automaticamente i singoli campioni e li inserisce nella sezione dedicata alle analisi.

3.6 Analisi

Il modulo Analisi gestisce la fase sperimentale e diagnostica sui campioni microbiologici generati dal sistema. Per poter essere analizzati, i campioni devono essere sottoposti a un periodo di incubazione a temperatura controllata che permette la crescita delle colonie batteriche, le quali possono poi essere quantificate. La durata di questo periodo dipende dal metodo di prova specifico utilizzato.

Una volta terminata l'incubazione, si procede alla conta delle colonie cresciute e all'individuazione del tipo di batterio. Questi dati vengono inseriti nel sistema, insieme alla documentazione fotografica, e viene indicato il nome del tecnico che si è occupato dell'analisi, garantendo così la rintracciabilità dell'intera procedura.

Il sistema consente di registrare i risultati relativi ai batteri Gram-negativi [25] e alla speciazione come *test* diagnostici aggiuntivi. A causa della loro complessa struttura cellulare, caratterizzata da una membrana esterna che ne aumenta la persistenza nell'ambiente e la resistenza agli agenti chimici, questi microrganismi costituiscono un fondamentale indicatore di contaminazione in ambito sanitario [26]. Per garantirne il monitoraggio epidemiologico, il *software* prevede un campo dedicato in cui l'operatore può inserire il numero di colonie riscontrate. Per la speciazione, cioè la divisione in vari tipi di microrganismi, il tecnico sceglie il tipo di microrganismo e indica al sistema se è stato trovato. In caso positivo, è possibile specificare l'esatta specie identificata e la quantità numerica espressa in Unità Formanti Colonia (CFU). Tutti questi dati vengono salvati nel sistema e resi immediatamente disponibili per la stesura del rapporto di prova finale.

3.7 Rapporti di prova

Il modulo Rapporti di prova gestisce la redazione e l'emissione del documento finale del laboratorio che riepiloga le informazioni sul campionamento e gli esiti delle analisi microbiologiche condotte. Il rapporto di prova si genera a partire

dalla selezione delle modalità operative. Al fine di ridurre gli errori manuali e velocizzare il processo, quasi tutte le informazioni necessarie vengono recuperate automaticamente dal sistema; l'unica eccezione riguarda le informazioni di legge nell'appendice, che l'operatore deve selezionare manualmente in base al caso specifico.

Una volta preparato il documento, il tecnico lo sottopone all'approvazione del direttore e del responsabile del laboratorio. Questi ultimi possono approvare il rapporto di prova, rendendolo pronto per la spedizione, oppure rifiutarlo, inserendo una motivazione. In caso di rifiuto, il documento ritorna ai tecnici che, dopo aver apportato le modifiche necessarie, possono inviare una nuova richiesta di approvazione.

La spedizione vera e propria al cliente viene eseguita dal responsabile dei sistemi informativi. Questa operazione attiva un blocco automatico e definitivo su tutte le informazioni associate al rapporto di prova, impedendo qualsiasi modifica successiva e garantendo l'integrità del dato. Se dopo il blocco ci si accorge di eventuali errori, il sistema non permette in alcun modo di sbloccare il documento originale. Per rimediare, è possibile creare una "copia correttiva", una funzione che consente di modificare i dati precedentemente bloccati, generando un nuovo documento e lasciando comunque intatto e storicizzato il rapporto di prova iniziale da cui è stata tratta la copia.

3.8 Sistema di logging

Il sistema di *audit logging* è progettato specificamente per garantire l'integrità dei dati, la responsabilità delle operazioni e il rispetto dei severi requisiti di tracciabilità imposti dall'ente di accreditamento Accredia. Il modulo ha il compito di monitorare e registrare in modo permanente qualsiasi operazione di creazione o modifica effettuata sugli oggetti del *database*. Per ogni evento, il sistema verifica e memorizza tre informazioni essenziali: l'identità dell'utente che ha compiuto l'azione (chi), il dettaglio dei campi modificati con i relativi valori (cosa) e la data e l'ora dell'operazione (quando).

Il sistema di *logging* è strutturato in cinque sezioni specifiche:

1. Liste e Anagrafica: monitora le variazioni dei dati di base del sistema. Il

sistema registra la storia di tutte le modifiche fatte nel tempo a ogni oggetto (per esempio un committente, una struttura o un lotto di consumo nelle liste).

2. Verballi: traccia l'intero ciclo di vita del documento operativo, registrando ogni modifica apportata alle informazioni logistiche, alle aree di campionamento o alle impostazioni iniziali del verbale specifico;
3. Analisi: registra in modo granulare tutte le modifiche effettuate sui singoli campioni biologici associati a un determinato verbale, monitorando l'inserimento o la revisione dei dati critici, come la conta delle colonie, i risultati dei test sui batteri Gram-negativi e i parametri di speciazione.
4. Rapporti di prova (*Test Report*): monitora la cronologia delle modifiche di ogni rapporto di prova specifico durante le fasi di redazione, richiesta di approvazione, rifiuto o emissione della copia correttiva.
5. Autenticazione (*Login*): gestisce il monitoraggio della sicurezza e degli accessi alla piattaforma. Questa sezione registra ogni evento di accesso andato a buon fine, ogni tentativo fallito e ogni operazione di *logout*. Oltre all'utente e all'orario, il sistema memorizza l'indirizzo IP del dispositivo mittente e l'interfaccia del *browser* (*user agent*) utilizzato per identificare l'origine dell'accesso.

Implementazione del progetto

Questo capitolo descrive gli aspetti implementativi del progetto sviluppato, illustrando le soluzioni adottate e le scelte tecniche effettuate durante la realizzazione del sistema. L'obiettivo è mostrare come i concetti teorici introdotti nei capitoli precedenti siano stati applicati concretamente per costruire un'applicazione funzionale, organizzata e facilmente estendibile.

In una prima fase si presentano le tecnologie utilizzate per lo sviluppo dell'applicazione e le motivazioni che hanno portato alla loro adozione. Di seguito si analizza la gestione del routing, approfondendo il sistema di utenti, ruoli e permessi, i meccanismi di autenticazione mediante middleware e l'organizzazione delle rotte applicative.

Si illustra quindi l'implementazione dell'architettura Model-View-Controller (MVC), descrivendo la suddivisione delle responsabilità tra i diversi componenti del sistema e il flusso di elaborazione delle richieste. A seguire, si espongono i principali automatismi sviluppati, tra cui la gestione dei campioni, la generazione automatica dei documenti PDF e i meccanismi di blocco dei rapporti di prova.

Infine, il capitolo descrive il processo di migrazione dei dati dalla prima alla seconda versione del sistema e le procedure di logging predisposte per il monitoraggio delle attività e la tracciabilità delle operazioni eseguite all'interno dell'applicazione.

La trattazione di questi argomenti consente di completare la descrizione del sistema, fornendo una visione complessiva delle scelte progettuali in vista delle considerazioni conclusive.

4.1 Tecnologie utilizzate

L'architettura *software* è stata interamente sviluppata utilizzando il *framework* Laravel (versione 11), un *framework* applicativo *web* basato su PHP (versione 8.2) che adotta il paradigma orientato agli oggetti e supporta le funzionalità moderne del linguaggio, quali i tipi di intersezione, le proprietà in sola lettura e la scomposizione dei costruttori.

L'applicazione si appoggia a un sistema di gestione di *database* relazionale (RDBMS) MySQL (versione 8.0), configurato per garantire l'integrità referenziale e la persistenza dei dati attraverso l'uso del motore di memorizzazione InnoDB, il quale supporta transazioni ACID e vincoli di chiave esterna (*foreign keys*).

La gestione delle dipendenze del *backend* è affidata al gestore di pacchetti Composer, il quale provvede all'*autoloader* delle classi secondo lo standard PSR-4. Parallelamente, l'ecosistema *front-end* è gestito tramite il *package manager* npm. Il processo di compilazione e raggruppamento degli *asset* (*bundling*) è demandato a Vite, integrato nel *framework* tramite il pacchetto ufficiale per Laravel. Per l'interfaccia utente è stato impiegato il *framework* Bootstrap, affiancato dal preprocessore CSS Sass e dal tema AdminBSB *Material Design* per fornire un *design* reattivo. L'infrastruttura *client-side* è ulteriormente supportata dalla libreria Axios per le comunicazioni HTTP asincrone e include i moduli per l'integrazione di componenti Vue.js.

Rispetto alle versioni precedenti, Laravel 11 introduce una struttura del progetto significativamente snellita: i *file* di configurazione generali sono stati accorpati e la registrazione dei *middleware* e delle rotte è stata centralizzata all'interno del *file* `bootstrap/app.php`, eliminando i tradizionali *file* *Kernel* HTTP e *Console*.

4.2 Routing

Il *framework* Laravel utilizza un sistema di *routing* che associa le richieste HTTP alle funzioni nei *controller*. Attraverso questo meccanismo, ogni richiesta viene instradata verso il componente responsabile della relativa elaborazione, consentendo una chiara separazione tra gestione delle richieste, logica applicativa e presentazione dei dati.

Nel progetto sviluppato il *routing* svolge inoltre un ruolo centrale nella gestione della sicurezza, poiché costituisce il punto in cui vengono applicati i controlli di autenticazione e autorizzazione. Le rotte sono organizzate in gruppi logici e protette mediante *middleware* dedicati, garantendo che le funzionalità del sistema siano accessibili esclusivamente agli utenti autorizzati.

Nei paragrafi successivi vengono descritti il modello di gestione dei ruoli e dei permessi adottato, i *middleware* utilizzati per il controllo degli accessi e l'organizzazione delle principali rotte applicative.

4.2.1 Utenti, ruoli e permessi

La gestione delle autorizzazioni è basata sul modello *Role-Based Access Control* (RBAC), implementato mediante il pacchetto *spatie/laravel-permission*. Tale approccio consente di separare l'identità dell'utente dai privilegi operativi assegnati, migliorando la flessibilità e la manutenibilità del sistema.

L'infrastruttura autorizzativa è composta da due livelli distinti: I ruoli, che definiscono il profilo generale dell'utente all'interno dell'applicazione e i permessi, che determinano le operazioni specifiche che l'utente può eseguire.

I principali ruoli applicativi sono: *admin*, *laboratory_director*, *laboratory_manager*, *management*, *technician*, *detector*, *visitor* e *computer_technician*. Tali ruoli identificano le diverse figure professionali coinvolte nei processi operativi del laboratorio.

Oltre ai ruoli, il sistema prevede un insieme di permessi granulari che consentono di controllare in modo puntuale l'accesso alle singole funzionalità. I permessi implementati sono riportati nella Tabella 4.1. Questa suddivisione consente di applicare il principio del minimo privilegio, assegnando a ciascun operatore esclusivamente le autorizzazioni necessarie allo svolgimento delle proprie attività. La verifica delle autorizzazioni avviene su due livelli differenti. I ruoli vengono con-

Permesso	Descrizione
<i>sampling</i>	Accesso alle attività di campionamento.
<i>analysis</i>	Gestione dei dati analitici e dei rapporti di prova.
<i>rdp_alidation</i>	Validazione dei rapporti di prova.
<i>Rdp_reopening</i>	Riapertura di rapporti precedentemente validati.
<i>data_edit</i>	Modifica delle anagrafiche e dei dati di configurazione.
<i>System_edit</i>	Amministrazione avanzata del sistema.
<i>rdp_shipping</i>	Gestione dell'invio dei rapporti di prova.
<i>viewing</i>	Accesso in sola lettura ai dati assegnati.

Tabella 4.1: Descrizione dei permessi granulari del sistema.

```
@can('rdp_reopening')
    <form action="{route('admin.testReport.revisionTestReport
                        $(testReport->id) } }"
        method="POST">
        @csrf
            <button type="submit">
                Revisione RDP
            </button>
        </form>
@endcan
```

Listing 4.1: Esempio della direttiva @can.

trollati principalmente durante la fase di instradamento delle richieste mediante il *middleware role*, che limita l'accesso alle diverse aree applicative. I permessi granulari vengono invece verificati all'interno delle viste *Blade* tramite la direttiva @can, che consente di mostrare o nascondere specifiche funzionalità in base alle autorizzazioni assegnate all'utente. Questo approccio permette di combinare un controllo degli accessi a livello di area applicativa con una gestione più fine delle singole operazioni disponibili all'interno dell'interfaccia.

Come mostrato nel Listato 4.1, il pulsante di revisione del Rapporto di Prova viene reso disponibile soltanto agli utenti in possesso del permesso *rdp_reopening*.

4.2.2 Middleware del login

I *middleware* rappresentano uno dei principali meccanismi di sicurezza del *framework* Laravel e consentono di intercettare le richieste HTTP prima che queste raggiungano i *controller* applicativi. Nel progetto sono stati utilizzati per implementare i controlli di autenticazione, autorizzazione e validità degli account utente.

L'accesso alle funzionalità riservate è subordinato al superamento di una catena di *middleware* eseguiti in sequenza. Ciascun componente è responsabile di una specifica verifica e può interrompere il flusso della richiesta qualora i requisiti di accesso non risultino soddisfatti. In particolare, il sistema utilizza tre *middleware* principali:

- *auth*: verifica che l'utente abbia effettuato correttamente l'accesso alla piattaforma. Le richieste provenienti da utenti non autenticati vengono reindirizzate alla pagina di *login*.
- *role*: fornito dal pacchetto *spatie/laravel-permission*, controlla che l'utente appartenga a uno dei ruoli autorizzati ad accedere alla specifica area applicativa.
- *status*: *middleware* personalizzato sviluppato nell'ambito del progetto. Ha il compito di verificare che l'account dell'utente sia attivo. Qualora un amministratore disabiliti un account già autenticato, il *middleware* invalida la sessione attiva e impedisce ulteriori accessi alle funzionalità del sistema.

L'utilizzo combinato di questi controlli consente di implementare una politica di sicurezza multilivello, nella quale autenticazione, ruolo e stato dell'utenza vengono verificati prima dell'esecuzione della logica applicativa.

Per quanto riguarda il *middleware* personalizzato *status* (Listato 4.2), la logica implementata verifica la presenza di una sessione autenticata e controlla il valore del campo *status* associato all'utente. In caso di account disabilitato, la sessione viene terminata e l'utente viene reindirizzato alla schermata di autenticazione.

```
if (auth()->check() && !auth()->user()->status) {  
    auth()->logout();  
    return redirect('login');  
}  
return $next($request);
```

Listing 4.2: Logica di validazione dello stato utente nel middleware personalizzato.

4.2.3 Rotte

Le rotte dell'applicazione sono definite nel *file* `routes/web.php` e costituiscono il punto di accesso alle funzionalità implementate nei *controller*. Per migliorare la manutenibilità del codice e centralizzare i controlli di sicurezza, le rotte sono organizzate in gruppi logici corrispondenti alle diverse aree operative del sistema.

La piattaforma è configurata come sistema ad accesso controllato. Le funzionalità di registrazione autonoma degli utenti, recupero della password e verifica dell'indirizzo *email* sono state disabilitate, poiché la gestione delle utenze viene effettuata esclusivamente dagli amministratori dell'applicazione.

L'*endpoint* principale dell'applicazione svolge una funzione di instradamento iniziale. Gli utenti non autenticati vengono reindirizzati alla schermata di *login*, mentre gli utenti autenticati vengono indirizzati verso l'area operativa associata al proprio ruolo.

Le funzionalità amministrative e operative sono raccolte all'interno di gruppi di rotte protette. A tali gruppi vengono applicati un prefisso URL comune (`/admin`), una convenzione uniforme per la denominazione delle rotte (`admin.*`) e i *middleware* responsabili dei controlli di autenticazione e autorizzazione.

Il Listato 4.3 mostra una versione semplificata della configurazione utilizzata per il gruppo principale di rotte amministrative:

4.3 MVC

L'applicazione è sviluppata secondo il *pattern* architetturale *Model-View-Controller* (MVC), esteso mediante l'introduzione di un *Service Layer* dedicato alla gestione della logica applicativa. Questa scelta progettuale consente di mantenere separa-

```
Route::middleware([
    'auth',
    'role:admin|(\textit{laboratory_director})|...',
    'status'
])->prefix('admin')
->name('admin.')
->group(function () {
    Route::get('/home', [HomeController::class, 'index'])
        ->name('home');

    // Ulteriori blocchi di rotte relativi ad altre risorse
});
```

Listing 4.3: Configurazione dei gruppi di rotte amministrative con applicazione di middleware.

ti i diversi livelli dell'applicazione: i *controller* gestiscono le richieste HTTP e le risposte verso il *client*, i modelli rappresentano le entità persistenti del dominio applicativo, mentre i servizi centralizzano le regole di *business* e le operazioni più complesse.

Per mostrare concretamente come i diversi componenti collaborino tra loro, viene analizzato il flusso di eliminazione di un verbale di campionamento. Il caso di studio consente di seguire l'intero ciclo di elaborazione di una richiesta, dalla visualizzazione dell'interfaccia utente fino alla persistenza delle modifiche sul *database*, evidenziando il ruolo svolto da modello, servizio, *controller* e vista.

Il primo elemento coinvolto nell'implementazione è il modello di dominio, responsabile della rappresentazione delle entità persistenti e della loro interazione con il *database*. Nel caso in esame, tale ruolo è svolto dalla classe *Verbal* nel [Listato 4.4](#), che rappresenta i verbali di campionamento memorizzati nella tabella SQL *verbals*. La classe utilizza alcuni *trait* forniti dal *framework* e dal progetto applicativo. In particolare, il *trait* *SoftDeletes* implementa la cancellazione logica dei *record* mediante la valorizzazione del campo *deleted_at*, preservando lo storico delle informazioni senza rimuoverle fisicamente dal *database*; invece, il *trait* personalizzato *EncryptedAttribute* consente la memorizzazione cifrata dei dati sensibili.

La classe definisce esplicitamente gli attributi `$fillable` per prevenire vulnerabilità di *Mass Assignment*, che si verificano quando i parametri di una richiesta HTTP vengono mappati automaticamente sugli attributi di un modello senza filtri preventivi, consentendo alterazioni del *payload* tramite l'iniezione di campi non autorizzati. La medesima classe implementa il metodo `casts()` per la tipizzazione forte dei dati in uscita dal *database* (es. conversione di stringhe SQL in oggetti *DateTime*) e l'albero delle relazioni *Eloquent* uno-a-molti (*HasMany*) e molti-a-uno (*BelongsTo*) che riflettono i vincoli relazionali del *database* MySQL. La classe *VerbalService* rappresenta il livello intermedio tra *controller* e modello, con il compito di centralizzare la logica applicativa associata alla gestione dei verbali. Questa scelta evita di inserire regole di *business* all'interno dei *controller* e favorisce il riutilizzo delle funzionalità in differenti contesti applicativi.

Come esempio viene analizzato il metodo *deleteVerbal* nel Listato 4.5, responsabile dell'eliminazione logica di un verbale di campionamento. Prima di eseguire l'eliminazione di un verbale, il servizio implementa un controllo di integrità referenziale logica: interroga il modello *VerbalRoom* per verificare la presenza di locali attivi collegati all'entità. Se la verifica rileva dipendenze, l'operazione viene interrotta restituendo un errore. In caso di esito positivo, il servizio aggiorna i campi di tracciabilità dell'*audit* (`delete_user_id` e `reason`), salva lo stato sul *database* ed esegue il *soft delete*, formalizzando l'*output* in una struttura *JsonResponse* standardizzata.

Dopo aver analizzato la logica applicativa implementata nel *Service Layer*, è possibile risalire al componente che ne determina l'esecuzione. L'invocazione del metodo *deleteVerbal()* avviene infatti all'interno del *controller*, che rappresenta il punto di ingresso delle richieste provenienti dal *client* e coordina l'interazione tra interfaccia utente e servizi applicativi.

Nel modulo in esame, il *VerbalController* riceve le richieste HTTP relative ai verbali di campionamento e delega l'esecuzione delle operazioni al *VerbalService*. Tale approccio mantiene il *controller* privo di logica di *business*, limitandone le responsabilità alla validazione dei dati ricevuti e all'orchestrazione del flusso applicativo. L'integrazione con il *VerbalService* avviene tramite il meccanismo della *Constructor Dependency Injection*, gestita dal *Service Container* di Laravel, che inserisce automaticamente l'istanza del servizio nel costruttore del *controller*.

Nel seguente esempio (Listato 4.6) viene analizzata l'azione REST *destroy*, uti-

```
class Verbal extends Model
{
    // Trait e interfacce di audit logging omesse in questa
    use (EncryptedAttribute), (HasFactory), (SoftDeletes);
    protected $fillable = [
        'campaign_id',
        'partition_id',
        // [...] Ulteriori attributi omessi per brevità
    ];
    protected function casts(): array
    {
        return [
            'revision_date' => 'datetime:Y-m-d',
            'sampling_date' => 'datetime:Y-m-d',
            // [...]
        ];
    }
    public function campaign(): BelongsTo
    {
        return $this->belongsTo(Campaign::class);
    }
    public function verbalRooms(): HasMany
    {
        return $this->hasMany(VerbalRoom::class);
    }
}
```

Listing 4.4: Implementazione del modello Verbal con trait, attributi protetti e relazioni.

lizzata per gestire la richiesta di eliminazione di un verbale. Laravel risolve automaticamente l'entità richiesta attraverso il meccanismo di *Implicit Model Binding*, mentre il *controller* recupera i dati necessari dalla richiesta HTTP e li inoltra al servizio competente.

```
class VerbalService
{
    public function deleteVerbal(Verbal $verbal, string $reason)
    {
        if ($this->hasRelatedRooms($verbal)) {
            return ErrorResponse();
        }

        $verbal->delete_user_id = Auth::id();
        $verbal->reason = $reason;

        $verbal->save();
        $verbal->delete();

        return successResponse();
    }
}
```

Listing 4.5: Implementazione della logica di eliminazione logica nel Service Layer.

L'implementazione evidenzia come il *controller* mantenga una struttura minimale e focalizzata esclusivamente sulla gestione della richiesta. Tutte le decisioni operative vengono demandate al *VerbalService*, ottenendo un codice più modulare, facilmente manutenibile e maggiormente testabile.

Dopo l'elaborazione della richiesta da parte del *controller* e del *service*, i dati vengono resi disponibili allo strato di presentazione, implementato mediante viste *Blade*. Nel seguito viene analizzato il file `show.blade.php`, responsabile della visualizzazione del dettaglio di un verbale di campionamento, del modulo Verbal come esempio rappresentativo del livello *View*.

La vista estende il *layout* comune dell'applicazione attraverso la direttiva `@extends`, garantendo uniformità grafica e riutilizzo degli elementi condivisi dell'interfaccia. I dati recuperati dal *controller* vengono resi disponibili alla vista. In particolare, il titolo del verbale viene generato dinamicamente tramite una logica di formattazione dedicata e memorizzata nella variabile `$nameVerbal`. Poiché tale

```
class VerbalController extends Controller
{
    protected VerbalService $verbalService;

    public function __construct(VerbalService $verbalService)
    {
        $this->verbalService = $verbalService;
    }

    public function destroy(Verbal $verbal, Request $request)
    {
        $reason = $request->input('reason');
        return $this->verbalService->deleteVerbal($verbal, $reason);
    }
}
```

Listing 4.6: Implementazione del Controller con Dependency Injection e azione REST destroy.

stringa include *tag* di formattazione HTML non persistiti nel *database*, si utilizza la sintassi `{!! $nameVerbal !!}`, che disabilita l'*escaping* automatico. La sicurezza del dato contro attacchi XSS è garantita dalla sanificazione preventiva operata a livello di *backend*.

Ai fini dell'esempio architetturale risulta particolarmente significativo il pulsante dedicato all'eliminazione del verbale (Listato 4.7). La vista si limita infatti a presentare l'elemento grafico e a rendere disponibile l'identificativo dell'entità selezionata, senza implementare alcuna logica applicativa relativa alla cancellazione.

L'interazione dell'utente con il pulsante viene gestita da uno *script* JavaScript (Listato 4.8) associato alla classe `btn-delete-verbal`. Lo *script* intercetta l'evento di *click*, richiede una conferma dell'operazione tramite una finestra modale e raccoglie la motivazione della cancellazione. Una volta confermata l'azione, viene inviata una richiesta AJAX di tipo DELETE all'*endpoint* esposto dal *controller*.

La vista e lo *script* JavaScript non eseguono direttamente alcuna operazione

```
@extends('layouts.app')

@section('content')
<div class="card">
<div class="header">
<p id="verbal_title">
{!! $nameVerbal !!}
</p>

<a href="javascript:void(0);" class="btn btn-light btn-del">
<i class="material-icons">delete</i>
</a>
</div>
</div>

@endsection
```

Listing 4.7: Struttura della vista Blade per la visualizzazione del dettaglio del verbale.

sul *database*, ma si limitano a raccogliere l'interazione dell'utente e ad inoltrarla ai livelli applicativi sottostanti. Il flusso della richiesta prosegue quindi nel *VerbalController*, che delega l'elaborazione al *VerbalService*, il quale applica le regole di *business* e interagisce con il modello *Verbal* tramite *Eloquent* ORM.

Questo esempio evidenzia la separazione delle responsabilità adottata nell'applicazione: la vista gestisce esclusivamente la presentazione e l'interazione con l'utente, il *controller* coordina la richiesta, il *service* implementa la logica applicativa e il modello si occupa della persistenza dei dati.

4.4 Automatismo

Gli automatismi implementati nell'applicazione supportano gli operatori nella gestione delle campagne di campionamento e delle analisi di laboratorio. Sono procedure *software* che si attivano automaticamente al verificarsi di specifiche condizioni o azioni, con lo scopo di ridurre le attività manuali, garantire la coerenza dei dati


```
$(document).on('click', '.btn-delete-verbal', function () {

    // Recupero dell'identificativo del verbale dall'elemento
    const verbalId = $(this).data('id');
    var url = $('#urls-verbal').data('url-destroy');

    // Intercettazione del flusso per l'acquisizione della
    Swal.fire({
        input: 'textarea'
    }).then((result) => {

        // Esecuzione condizionata alla conferma dell'utente
        if (result.isConfirmed) {

            // Inizializzazione della richiesta AJAX
            $.ajax({
                url: url,
                type: 'DELETE',
                data: {
                    reason: result.value // Motivo della cancellazione
                }
            });
        }
    });
});
```

Listing 4.8: Gestione dell'evento click e sottomissione della richiesta AJAX tramite jQuery.

e applicare uniformemente le regole di *business*. Intervengono in momenti chiave come la generazione e sincronizzazione dei campioni, la produzione di documentazione ufficiale in formato PDF e la gestione della validazione dei rapporti di prova. Questi meccanismi centralizzano la logica applicativa, riducono il rischio di errore

umano e mantengono le informazioni nel *database* sempre aggiornate rispetto allo stato reale delle attività.

4.4.1 Campioni

La generazione e la sincronizzazione dei campioni avvengono tramite un automatismo applicativo che si attiva durante la convalida o l'annullamento della convalida del verbale (Listato 4.9). Questo processo ha l'obiettivo di allineare la tabella dei campioni con lo stato reale della struttura gerarchica ($Verbal \rightarrow VerbalRoom \rightarrow VerbalRoomModality \rightarrow VerbalRoomModalityPoint$).

L'intera operazione viene eseguita all'interno di una transazione *database*, garantendo che tutte le modifiche vengano applicate in modo coerente. In caso di errore, il sistema annulla automaticamente le operazioni effettuate, evitando situazioni di inconsistenza tra i dati. Il sistema applica una logica idempotente, ovvero progettata per produrre sempre lo stesso risultato finale indipendentemente dal numero di esecuzioni consecutive.

L'automatismo può operare secondo due modalità differenti: convalida oppure annullamento della convalida. Nel caso di convalida, per ogni punto di campionamento vengono innanzitutto determinati i codici campione che dovrebbero essere presenti in base alla configurazione corrente. Successivamente vengono recuperati eventuali campioni precedentemente eliminati tramite *soft-delete*.

Se alcuni dei campioni attesi risultano già presenti nello storico, questi vengono ripristinati anziché essere ricreati. In questo modo viene preservata la continuità dei dati e viene evitata la generazione di duplicati. Infine, il sistema confronta l'elenco dei campioni attesi con quelli già esistenti e crea esclusivamente quelli mancanti. Invece quando la convalida viene revocata, il sistema esegue l'operazione inversa. I campioni associati ai punti di campionamento non vengono eliminati fisicamente dal *database*, ma sottoposti a *soft-delete*.

4.4.2 PDF

Il sistema gestisce l'emissione dei documenti formali attraverso moduli *software* deputati alla generazione di *report* in formato non modificabile (PDF). Il processo trasforma i dati relazionali memorizzati nel *database* in un documento statico,

```
public function convalidateAllRooms(verbalId, newState):
    Transazione.Esegui(
        Per ogni Room associata a verbalId:
            Room.isCompleted = newState
        Per ogni Modality associata a Room:
            Se Modality NON e' bloccata:
                Modality.isCompleted = newState
            Per ogni Point associato a Modality:
                Point.isCompleted = newState
                Se newState e' VERO:
                    // Fase di Convalida
                    expectedSamples =
                        generaLabel(Point)
                    ripristinaCampioni(expectedSamples)
                    creaCampioniMancanti(expectedSamples)

                Altrimenti:
                    // Fase di Annullamento

                    markDeletionInformation(Point)
                    softDeleteSamples(Samples)

    ) // Il framework gestisce Commit e Rollback
        automaticamente
```

Listing 4.9: Pseudocodice del processo di generazione e sincronizzazione dei campioni.

archiviato fisicamente e reso disponibile all'utente. L'architettura di esportazione è supportata dalla libreria *barryvdh/laravel-dompdf*.

Poiché la generazione dei documenti avviene in molteplici moduli dell'applicativo, viene analizzata a titolo esemplificativo la procedura relativa al verbale di ispezione. Il flusso di esecuzione si articola nelle seguenti fasi:

- **Ricostruzione del Grafo dei Dati:** A seguito della richiesta, il sistema

interroga il *database* per aggregare le informazioni associate al verbale, includendo i metadati della campagna, l'anagrafica del cliente, la gerarchia dei locali e le specifiche delle apparecchiature.

- **Rendering:** Il *set* di dati viene iniettato all'interno di una vista predefinita, incaricata di organizzare le informazioni secondo il *layout* della modulistica aziendale.
- **Estrazione e Archiviazione Dinamica:** Il documento viene generato in memoria e il relativo flusso binario viene estratto. Il sistema calcola un percorso di destinazione univoco basato sullo stato del campionamento.
- **Persistenza e Allineamento:** Il *file* viene scritto fisicamente nello *storage* dell'applicazione. Contestualmente, il nome del *file* viene persistito nel *record* del *database* relazionale per mantenere la consistenza dello stato.
- **Trasmissione:** Il sistema restituisce il flusso dati al *browser* del *client*.

Il Listato 4.10 mostra, a titolo di esempio, la generazione del PDF del verbale interno.

```
Se (verbal.type e' uguale a internal):
    datiContesto = recuperaGrafoDatiVerbale(verbalId)
    pdf = PDF::loadView('admin.pdf.internal_verbal_pdf',
        datiContesto)
    pdfContent = pdf->output()

    prefisso =
        determinaPrefissoStatoCampionamento(verbal.post_sampling)
    nomeFile = generaCodiceUnivoco(verbal.id) + ".pdf"
    percorso = calcolaPercorsoStorage(campaign.id, verbal.id,
        prefisso, nomeFile)

    Storage::put(percorso, pdfContent)
    verbal.update([
        'filename' => nomeFile
    ])

    Ritorna pdf->stream(nomeFile)
```

Listing 4.10: Pseudocodice del processo di rendering, archiviazione e trasmissione del verbale in formato PDF.

4.4.3 Blocco del rapporto di prova

L'ultimo automatismo analizzato riguarda il processo di verifica del Rapporto di Prova (RDP), documento che raccoglie e formalizza i risultati delle analisi eseguite sui campioni acquisiti durante le attività di campionamento.

L'obiettivo principale di questo automatismo è garantire che, una volta completato il processo di approvazione, i dati utilizzati per la redazione del rapporto non possano essere modificati accidentalmente, preservando la coerenza tra il documento emesso e le informazioni archiviate nel sistema.

Il processo viene attivato da un utente autorizzato durante la fase di verifica del rapporto e prevede due possibili esiti: approvazione oppure richiesta di revisione.

Nel caso di approvazione, il sistema aggiorna lo stato del Rapporto di Prova marcandolo come verificato e bloccato. Contestualmente, il vincolo di blocco viene propagato a tutti i campioni associati al rapporto. Questa operazione garantisce che i dati analitici utilizzati per la generazione del documento rimangano immutati dopo la validazione, evitando situazioni in cui successive modifiche ai campioni possano rendere non coerente la documentazione già approvata. Inoltre, il sistema registra il messaggio di verifica e l'identità dell'utente che ha effettuato l'operazione, consentendo la ricostruzione dello storico delle approvazioni.

Qualora il rapporto non venga approvato, il sistema attiva una procedura di revisione. In questo caso vengono annullati gli stati di validazione precedentemente assegnati e il documento viene nuovamente reso modificabile dagli operatori competenti. Infine, viene mandata una *e-mail* ai tecnici contenente le motivazioni del rifiuto, permettendo ai soggetti coinvolti di individuare le anomalie riscontrate e procedere con le necessarie correzioni.

4.5 Migrazione dati dalla versione 1 alla versione 2

4.6 Logging

Al fine di garantire la tracciabilità delle operazioni effettuate dagli utenti e soddisfare i requisiti di controllo e sicurezza, è stato implementato un sistema di *Audit*

```
Procedura VerificaRapportoDiProva(richiesta, testReportId):
    rapporto = RecuperaRapporto(testReportId)
    azione = richiesta.OttieniParametro('action')
    motivo = richiesta.OttieniParametro(
        'reason_verification')
    RegistraMessaggio(motivo, UtenteAttuale.id, rapporto.id)
    Se azione e' uguale a 'validate':
        rapporto.is_verified = VERO
        rapporto.is_blocked = VERO
        rapporto.SalvaSuDB()
        Sample.FiltraPerGerarchiaRapporto(rapporto.id)
        .AggiornaMassivamente(is_blocked=VERO)
    Altrimenti:
        Mail.InviaNotificaRifiuto(rapporto, azione,
            motivo, UtenteAttuale)
        rapporto.first_validation = FALSO
        rapporto.is_validated_laboratory_director = NULL
        rapporto.is_validated_laboratory_manager = NULL
        rapporto.SalvaSuDB()
    Fine Se
    Ritorna ReindirizzaAllaVista('admin.testReport.show',
        rapporto.id)
```

Listing 4.11: Pseudocodice della procedura di verifica e gestione degli stati del Rapporto di Prova.

Logging. Tale meccanismo consente di registrare sia gli eventi di autenticazione degli utenti sia le modifiche apportate ai dati.

L'infrastruttura è stata progettata in modo da risultare indipendente dalla logica di *business* dell'applicazione. In questo modo, le operazioni di registrazione vengono eseguite automaticamente senza introdurre codice aggiuntivo all'interno dei *controller* o dei servizi responsabili delle funzionalità principali.

Il sistema può essere suddiviso in due componenti principali:

- tracciamento degli eventi di autenticazione;
- tracciamento delle operazioni CRUD sui modelli applicativi.

La registrazione degli eventi di accesso è affidata alla classe *LoginAuditSubscriber*, che sfrutta il sistema di eventi fornito dal *framework* Laravel per intercettare le principali operazioni di autenticazione. In particolare, vengono monitorati:

- accesso effettuato con successo (*Login*);
- tentativo di accesso fallito (*Failed*);

- disconnessione dell'utente (*Logout*).

Per ciascun evento vengono raccolte e memorizzate alcune informazioni utili all'attività di *audit*, tra cui l'identificativo dell'utente, l'indirizzo IP e il *browser* utilizzato.

La persistenza dei dati viene delegata al servizio *AuditService*, responsabile della registrazione effettiva nella tabella *login_audits*.

Nella gestione dei tentativi falliti (*Failed*), il sistema implementa un controllo preventivo di esistenza dell'istanza utente. Se un utente tenta l'accesso con credenziali errate utilizzando un nome utente non censito, il sistema cattura l'anomalia gestendo il valore come *null*, registrando l'evento di attacco o errore senza sollevare eccezioni di *runtime*.

Listing 4.12 mostra l'implementazione del *subscriber*.

Per registrare le modifiche apportate alle entità persistenti dell'applicazione è stata implementata un'infrastruttura di *audit* basata sul *pattern Observer*. L'obiettivo è quello di automatizzare la registrazione delle operazioni di creazione, modifica ed eliminazione dei *record*, evitando di inserire logiche di *auditing* all'interno dei *controller* o dei servizi applicativi.

La soluzione adottata si basa su tre componenti principali:

- l'interfaccia `AuditableContract`;
- il *trait* `HasAudit`;
- l'*observer* `AuditObserver`.

L'interazione tra tali componenti consente di applicare le funzionalità di *audit* a qualsiasi modello dell'applicazione in maniera uniforme e riutilizzabile.

Il primo elemento dell'architettura è rappresentato dall'interfaccia `AuditableContract` nel Listato 4.13, che definisce il comportamento minimo richiesto ai modelli soggetti a tracciamento. Attraverso questo contratto ogni entità fornisce le informazioni necessarie al sistema di *audit*, tra cui:

- la tabella di destinazione nella quale salvare i *log*;
- l'eventuale identificativo dell'entità principale a cui associare il *log*;
- la necessità di registrare o meno il dettaglio delle modifiche effettuate.

```

class LoginAuditSubscriber
{
    protected AuditService $auditService;
    public function __construct(AuditService
        $auditService)
    {
        $this->auditService = $auditService;
    }
    public function handleUserLogin(Login $event):
        void
    {
        $this->auditService->logAuthEvent($event->user->getAuthIdentifier(),
            'login');
    }
    public function handleUserFailed(Failed $event):
        void
    {
        $userId = $event->user ? $event->user->id
            : null;
        $this->auditService->logAuthEvent($event->user?->getAuthIdentifier(),
            'failed');
    }
    public function handleUserLogout(Logout $event):
        void
    {
        $userId = $event->user ?
            $event->user->getAuthIdentifier() :
            null;
        $this->auditService->logAuthEvent($userId,
            'logout');
    }
    public function subscribe(Dispatcher $events):
        void
    {
        $events->listen(Login::class,
            [self::class, 'handleUserLogin']);
        $events->listen(Failed::class,
            [self::class, 'handleUserFailed']);
        $events->listen(Logout::class,
            [self::class, 'handleUserLogout']);
    }
}

```

Listing 4.12: Implementazione della classe LoginAuditSubscriber per il tracciamento degli eventi di autenticazione.

L'utilizzo di un'interfaccia permette di separare la logica di *auditing* dalle specifiche implementazioni dei modelli, favorendo l'estensibilità del sistema e garantendo un comportamento coerente tra tutte le entità monitorate.


```
interface AuditableContract
{
    public function getAuditTableName(): string;

    public function getAuditRootId(): ?int;

    public function requiresAuditPayload(): bool;
}
```

Listing 4.13: Definizione dell'interfaccia AuditableContract.

```
trait HasAudit
{
    public static function bootHasAudit(): void
    {
        static::observe(AuditObserver::class);
    }

    public function getAuditTableName(): string
    {
        return 'list_audits';
    }

    public function getAuditRootId(): ?int
    {
        return null;
    }

    public function requiresAuditPayload(): bool
    {
        return true;
    }
}
```

Listing 4.14: Implementazione del trait HasAudit e dell'automazione di boot per l'osservazione.

Per evitare la duplicazione di codice è stato realizzato il *trait* `HasAudit`, che fornisce un'implementazione predefinita dei metodi definiti dall'interfaccia. L'adozione di questo approccio consente ai modelli di ottenere immediatamente il supporto all'*audit* semplicemente importando il *trait*, senza dover riscrivere la stessa logica in più classi. Oltre a fornire i valori di *default*, il *trait* registra automaticamente l'*observer* responsabile del monitoraggio delle modifiche, semplificando ulteriormente l'integrazione del sistema di *auditing*. Il Listato 4.14 riporta l'implementazione del *trait*.

La registrazione effettiva delle operazioni viene gestita dalla classe `AuditObserver`,

```
class AuditObserver
{
    protected AuditService $auditService;

    public function __construct(AuditService $auditService)
    {
        $this->auditService = $auditService;
    }

    public function created(AuditableContract $model): void
    {
        $this->auditService->logEvent($model, 'created');
    }

    public function updated(AuditableContract $model): void
    {
        if ($model->wasChanged('deleted_at') &&
            count($model->getChanges()) === 1) {
            return;
        }
        $this->auditService->logEvent($model, 'updated');
    }

    public function forceDeleted(AuditableContract $model): void
    {
        $this->auditService->logEvent($model, 'deleted');
    }
}
```

Listing 4.15: Classe AuditObserver per la centralizzazione degli eventi legati alla persistenza.

che intercetta gli eventi generati dai modelli durante il loro ciclo di vita. In particolare, vengono monitorate le seguenti operazioni:

- creazione di un nuovo *record* (*created*);
- modifica di un *record* esistente (*updated*);
- eliminazione permanente di un *record* (*forceDeleted*).

Quando uno di questi eventi si verifica, l'*observer* delega la registrazione al servizio *AuditService*, responsabile della costruzione e della persistenza del *log*. L'implementazione completa è strutturata nel Listato 4.15.

Durante la fase di aggiornamento è stato introdotto un controllo aggiuntivo per distinguere le normali modifiche dalle operazioni di cancellazione logica (*soft*

delete). In questo modo vengono evitati *log* duplicati e viene mantenuta una cronologia più chiara delle operazioni effettuate sugli oggetti applicativi. L'utilizzo del *pattern Observer* permette inoltre di centralizzare la logica di *auditing* in un unico componente, riducendo l'accoppiamento con il resto dell'applicazione e facilitando eventuali estensioni future.

La classe *AuditService* rappresenta il punto centrale dell'intero sistema di *auditing*. Tutti gli eventi rilevati dall'applicazione convergono infatti verso questo componente, indipendentemente dalla loro origine.

In particolare, il servizio riceve informazioni da due sorgenti distinte:

- *LoginAuditSubscriber*, responsabile del tracciamento degli eventi di autenticazione;
- *AuditObserver*, incaricato di monitorare le operazioni CRUD sui modelli applicativi.

Questa scelta progettuale consente di centralizzare la logica di registrazione degli eventi, evitando duplicazioni di codice e mantenendo separata la fase di intercettazione degli eventi dalla loro effettiva persistenza nel *database*.

Per gli eventi di autenticazione il servizio utilizza il metodo `logAuthEvent()` nel Listato 4.16, che registra nella tabella `login_audits` l'identificativo dell'utente, il tipo di evento, l'indirizzo IP del *client* e il *browser* utilizzato. Queste informazioni consentono di mantenere uno storico degli accessi alla piattaforma e di individuare eventuali tentativi di autenticazione non autorizzati.

Per le operazioni CRUD viene invece utilizzato il metodo `logEvent()` nel Listato 4.17, responsabile della costruzione del *record* di *audit* associato alle entità applicative. Il metodo si occupa di strutturare le informazioni e memorizzarle in formato JSON, applicando specifici criteri di sanitizzazione e ottimizzazione delle risorse:

- **Filtro dei metadati di sistema:** I campi tecnici legati alla gestione del *database* (es. `id`, `created_at`, `updated_at`, `deleted_at`) e gli attributi contenenti chiavi o *hash* crittografici (es. `password`) vengono esclusi dal *payload* per ragioni di sicurezza e per limitare l'occupazione di memoria su MySQL.
- **Isolamento delle sole modifiche effettive:** In fase di *update*, il servizio esegue un'intersezione logica tra lo stato originario del modello e l'insieme

```

/**
 * Registra un evento di autenticazione nella tabella login_audits.
 */
public function logAuthEvent(?int $userId, string $eventType):
    void
{
    DB::table('login_audits')->insert([
        'user_id'      => $userId,
        'event_type'   => $eventType,
        'ip_address'   => request()->ip(),
        'user_agent'   => request()->userAgent(),
        'created_at'   => now(),
    ]);
}

```

Listing 4.16: Registrazione di un evento di autenticazione nella tabella `login_audits`.

degli attributi che hanno subito una mutazione. Ciò consente di popolare il campo `old_values` unicamente con lo storico dei campi alterati, escludendo i valori rimasti invariati. Lo stato successivo viene analogamente ridotto e inserito nel campo `new_values`.

- **Tracciamento Gerarchico (`root_id`):** Qualora il modello implementi una struttura gerarchica complessa, il metodo `getAuditRootId()` restituisce l'identificativo dell'entità radice (es. il modulo documentale principale). Questa associazione semplifica la successiva ricostruzione cronologica di un intero blocco di dati correlati mediante l'esecuzione di *query* indicizzate.

Il funzionamento polimorfico dell'architettura permette di estendere il sistema sia a modelli strutturati con vincoli gerarchici, sia a entità standard prive di relazioni complesse.

Alcune entità rappresentano elementi centrali del dominio applicativo e necessitano di una gestione dedicata dei *log*. In questi casi il modello ridefinisce i metodi forniti dal *trait* per specificare una tabella di *audit* dedicata e un identificativo gerarchico utilizzato per il raggruppamento degli eventi. Un esempio è rappresentato dal modello *Verbal* nel Listato 4.18.

In questo caso tutti gli eventi associati al verbale vengono salvati nella tabella `verbal_audits`. Inoltre il valore restituito da `getAuditRootId()` consente di associare eventuali modifiche effettuate sulle entità correlate allo stesso verbale

```
public function logEvent(AuditableContract $model, string
    $eventType): void
{
    // 1. Verifica dei requisiti del contratto
    if (!$model->requiresAuditPayload()) {
        return;
    }

    // 2. Estrazione e isolamento delle sole variazioni
    (Delta)
    // I metodi interni si occupano di rimuovere password e
    metadati (es. created_at)
    $oldValues = $this->extractFilteredOldValues($model,
        $eventType);
    $newValues = $this->extractFilteredNewValues($model,
        $eventType);

    // 3. Costruzione del payload relazionale e gerarchico
    $payload = [
        'user_id'      => Auth::id(),
        'event_type'   => $eventType,
        'old_values'   => json_encode($oldValues),
        'new_values'   => json_encode($newValues),
        'root_id'      => $model->getAuditRootId(),
        // ... [altri campi identificativi del polimorfismo
        //       omessi per brevità]
    ];

    // 4. Persistenza dinamica sulla tabella definita dal
    contratto
    DB::table($model->getAuditTableName())->insert($payload);
}
```

Listing 4.17: Metodo `logEvent` per la strutturazione e persistenza del payload di audit.

principale. Questa soluzione semplifica la consultazione dello storico delle operazioni e permette di ricostruire in modo immediato tutte le modifiche relative a uno specifico documento.

Per le entità che non richiedono particolari personalizzazioni è sufficiente utilizzare il comportamento predefinito fornito dal *trait*. Un esempio è rappresentato dal modello *Device* nel Listato 4.19. In questo caso il sistema utilizza automaticamente i valori di *default* definiti nel *trait*:

- i *log* vengono salvati nella tabella `list_audits`;
- non viene associato alcun identificativo gerarchico;

```
class Verbal extends Model implements AuditableContract
{
    use HasAudit, EncryptedAttribute, HasFactory, SoftDeletes;

    public function getAuditTableName(): string
    {
        return 'verbal_audits';
    }

    public function getAuditRootId(): ?int
    {
        return $this->id;
    }
}
```

Listing 4.18: Personalizzazione del contratto di audit nel modello Verbal.

- viene mantenuto il tracciamento delle modifiche tramite il calcolo del delta.

Grazie a questo approccio l'integrazione del sistema di *audit* richiede un numero minimo di modifiche al modello e non comporta la duplicazione della logica di registrazione.

```
class Device extends Model implements AuditableContract
{
    use HasAudit; // [...] Ulteriori trait omessi per brevit 
}
```

Listing 4.19: Implementazione predefinita del contratto di audit nel modello Device.

Conclusione

Bibliografia

- [1] *HTML Standard*. URL: <https://html.spec.whatwg.org/multipage/introduction.html#introduction> (visitato il giorno 09/04/2026).
- [2] Jennifer Niederst Robbins. *Learning Web Design: A Beginner's Guide to HTML, CSS, Javascript, and Web Images*. Sixth edition. Santa Rosa, CA: O'Reilly, 2025. 1 p. ISBN: 978-1-0981-3768-7.
- [3] *Cascading Style Sheets, Designing for the Web – Chapter 2: CSS*. URL: <https://www.w3.org/Style/LieBos2e/enter/> (visitato il giorno 10/04/2026).
- [4] *Get Started with Bootstrap · Bootstrap v5.2*. URL: <https://getbootstrap.com/docs/5.2/getting-started/introduction/> (visitato il giorno 10/04/2026).
- [5] Ecma International. *ECMA-262, 16th Edition, June 2025*. Language Specification. Ver. 16th Edition. Geneva, giu. 2025. URL: https://ecma-international.org/wp-content/uploads/ECMA-262_16th_edition_june_2025.pdf (visitato il giorno 10/04/2026). Standard.
- [6] David Flanagan. *JavaScript: The Definitive Guide; Master the World's Most-Used Programming Language*. Seventh edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly, 2020. 1 p. ISBN: 978-1-4919-5200-9 978-1-4919-5198-9.
- [7] OpenJS Foundation- openjsf.org. *jQuery API Documentation*. 17 Mag. 2026. URL: <https://api.jquery.com/> (visitato il giorno 17/05/2026).

- [8] James F. Kurose e Keith W. Ross. *Computer Networking: A Top-down Approach*. Eighth edition. Hoboken: Pearson, 2021. ISBN: 978-0-13-668155-7.
- [9] *Usage Statistics and Market Share of Server-side Programming Languages for Websites, May 2026*. URL: https://w3techs.com/technologies/overview/programming_language (visitato il giorno 08/05/2026).
- [10] *PHP: History of PHP - Manual*. URL: <https://www.php.net/manual/en/history.php.php> (visitato il giorno 08/05/2026).
- [11] *PHP: Preface - Manual*. URL: <https://www.php.net/manual/en/index.php> (visitato il giorno 08/05/2026).
- [12] *Composer*. URL: <https://getcomposer.org/doc/> (visitato il giorno 08/05/2026).
- [13] *The State of PHP 2025 – Expert Review*. URL: <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/#php-frameworks-and-cms> (visitato il giorno 18/05/2026).
- [14] *Installation | Laravel 13.x - The Clean Stack for Artisans and Agents*. URL: <https://laravel.com/docs/13.x/installation> (visitato il giorno 25/05/2026).
- [15] *Controllers*. URL: <https://laravel.com/docs/13.x/controllers> (visitato il giorno 25/05/2026).
- [16] *Views*. URL: <https://laravel.com/docs/13.x/views> (visitato il giorno 25/05/2026).
- [17] *Eloquent*. URL: <https://laravel.com/docs/13.x/eloquent> (visitato il giorno 25/05/2026).
- [18] *Introduction to Laravel and MVC Framework - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/php/introduction-to-laravel-and-mvc-framework/> (visitato il giorno 30/05/2026).
- [19] *Request Lifecycle | Laravel 13.x - The Clean Stack for Artisans and Agents*. Laravel. URL: <https://laravel.com/docs/13.x/lifecycle> (visitato il giorno 30/05/2026).
- [20] *Service Providers | Laravel 13.x - The Clean Stack for Artisans and Agents*. Laravel. URL: <https://laravel.com/docs/13.x/providers> (visitato il giorno 30/05/2026).

-
- [21] *Middleware / Laravel 13.x - The Clean Stack for Artisans and Agents*. Laravel. URL: <https://laravel.com/docs/13.x/middleware> (visitato il giorno 30/05/2026).
- [22] *Routing / Laravel 13.x - The Clean Stack for Artisans and Agents*. Laravel. URL: <https://laravel.com/docs/13.x/routing> (visitato il giorno 30/05/2026).
- [23] *Service Container / Laravel 13.x - The Clean Stack for Artisans and Agents*. Laravel. URL: <https://laravel.com/docs/13.x/container> (visitato il giorno 30/05/2026).
- [24] *Your Health Is Our Job*. URL: <https://cias-ferrara.it/> (visitato il giorno 11/06/2026).
- [25] Junio Oliveira e Wanda C. Reygaert. “Gram-Negative Bacteria”. In: *StatPearls*. Treasure Island (FL): StatPearls Publishing, 2026. PMID: [30855801](https://pubmed.ncbi.nlm.nih.gov/30855801/). URL: <http://www.ncbi.nlm.nih.gov/books/NBK538213/> (visitato il giorno 11/06/2026).
- [26] *WHO Bacterial Priority Pathogens List 2024: Bacterial Pathogens of Public Health Importance, to Guide Research, Development, and Strategies to Prevent and Control Antimicrobial Resistance*. 1st ed. Geneva: World Health Organization, 2024. 1 p. ISBN: 978-92-4-009346-1.